Introduction to Audio Domain:
Create Your Own Intelligent Voice Assistant

Presented by Petr Grinberg
HSE, 2026
Based on HSE DLA Course: https://github.com/markovka17/dla

Welcome! I am Petr

- ***MSc EPFL, LauzHack Mentor***
- ***Lecturer at NRU HSE***
- ***Sony***
- ***Ex. LCAV EPFL***
- ***Ex. Reality Defender Inc.***
- ***Ex. Samsung R&D***

Follow me on GitHub (@Blinorot)

Plan of the workshop

1. Deep Learning for Audio Processing
    a. A bit of history
    b. What can we do with it?
2. Signal Processing Basics
3. The core parts of a Voice Assistant
    a. KWS
    b. ASR
    c. TTS
    d. LLM
4. The code for all parts:
    a. Vanilla PyTorch
    b. Pre-trained models
    c. APIs

Slides

A short history of Speech Recognition

50's

In **1952**, *Bell Laboratories* designed the “**Audrey**” system which could recognize a single voice speaking **digits** aloud

In **1962,** *IBM* introduced “**Shoebox**” which understood and responded to **16 words** in English.

60's

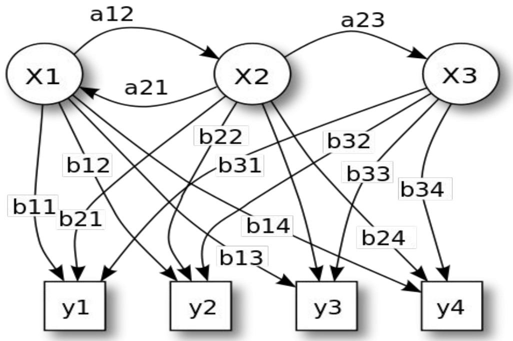
A short history of Speech Recognition

70's

DARPA's system was capable of understanding over **1,000** words. **Siri** was a spin-out of DARPA development :)

80's

The '80s saw speech recognition vocabulary go from a few hundred words to **several thousand words** thanks to **HMM**



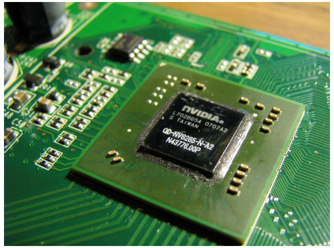
A short history of Speech Recognition

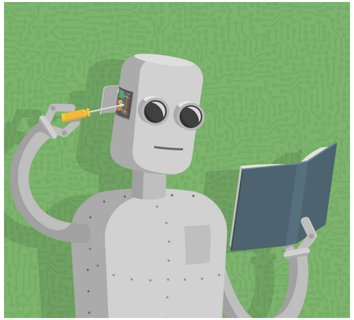
90's

Speech recognition was propelled forward in the 90s in large part because of **faster processors**

00-10's

And then came the era of big data, machine learning and GPUs





A short history of Speech Synthesis

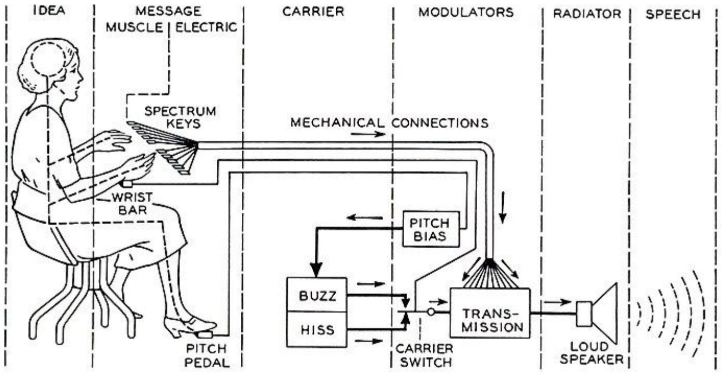


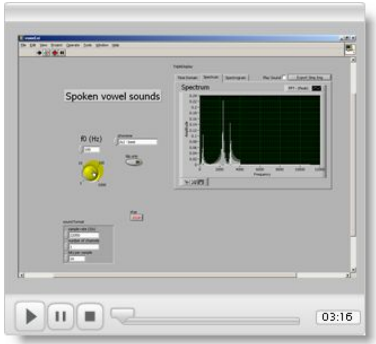
Fig. 8—Schematic circuit of the voder.

30's

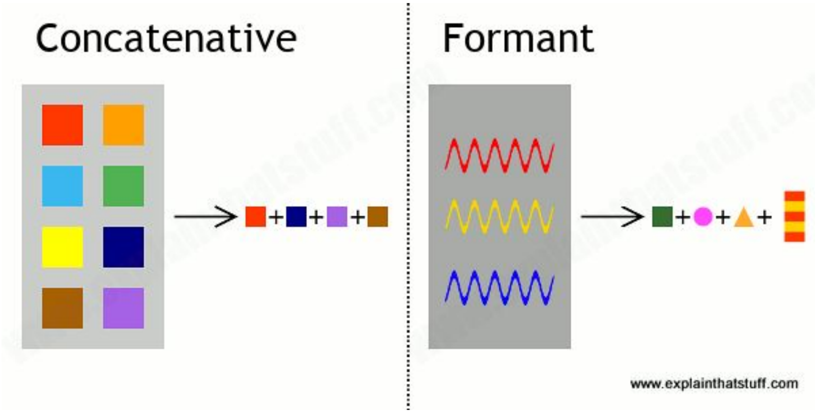
until 80's

In **1939**, *The Bell Laboratory*'s **Voder** was the first attempt to electronically synthesize human speech by breaking it down into its **acoustic components**

Formant-based on rules. You may listen examples in Atari&Sega games :)



A short history of Speech Synthesis

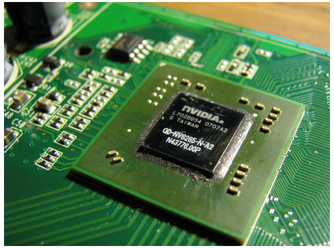


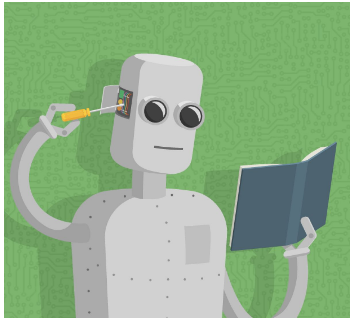
90-00's

Concatenative synthesis is a technique for synthesising sounds by concatenating short samples of recorded sound (called *units*).

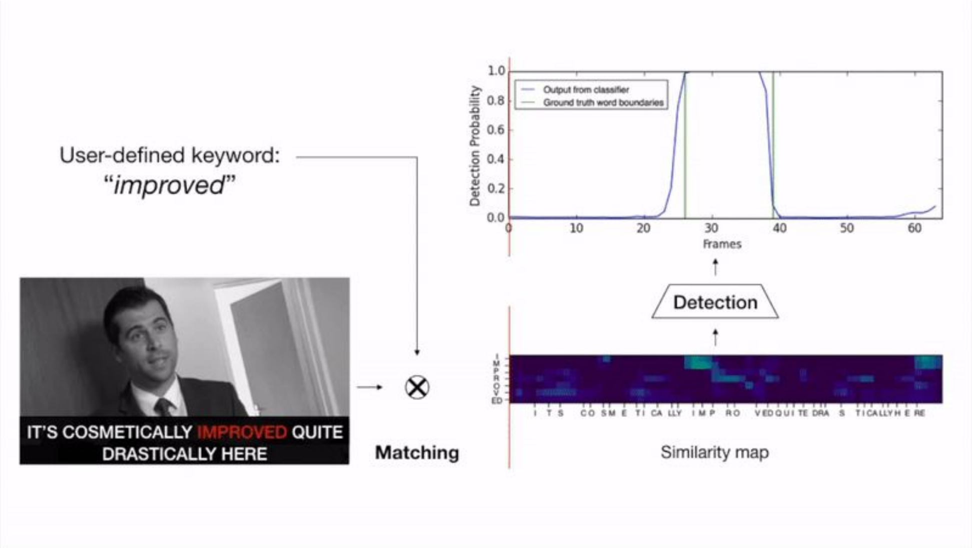
10's

And then came the era of big data, machine learning and GPUs



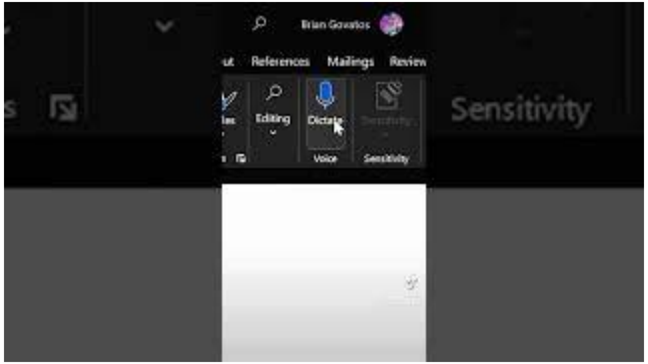


Tasks: Keyword Spotting (KWS)



Momeni, Liliane, et al. "Seeing wake words: Audio-visual keyword spotting." arXiv preprint arXiv:2009.01225 (2020).

Tasks: Automatic Speech Recognition



Voice Dictation

Meeting Summary

Speech to Text

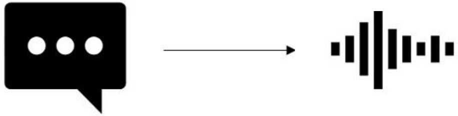
And, of course, voice assistants

Tasks: Text To Speech And Voice Conversion

Audio (Audio-Visual) Deepfakes*

AI Covers

Text to Speech



And, of course, voice assistants

*SadTalker + XTTS

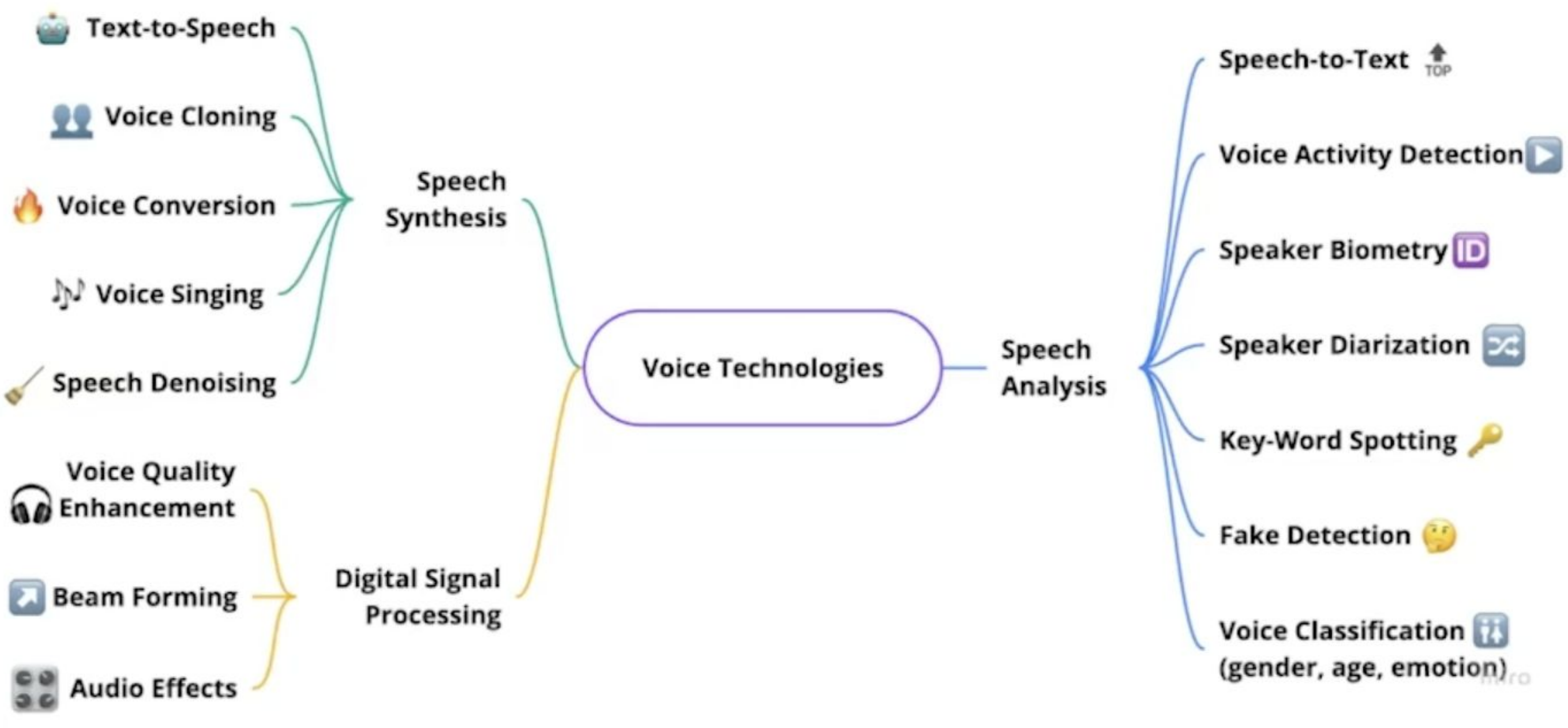
Tasks: Video To Audio

https://deepmind.google/discover/blog/generating-audio-for-video/

Tasks: Sketch To Audio

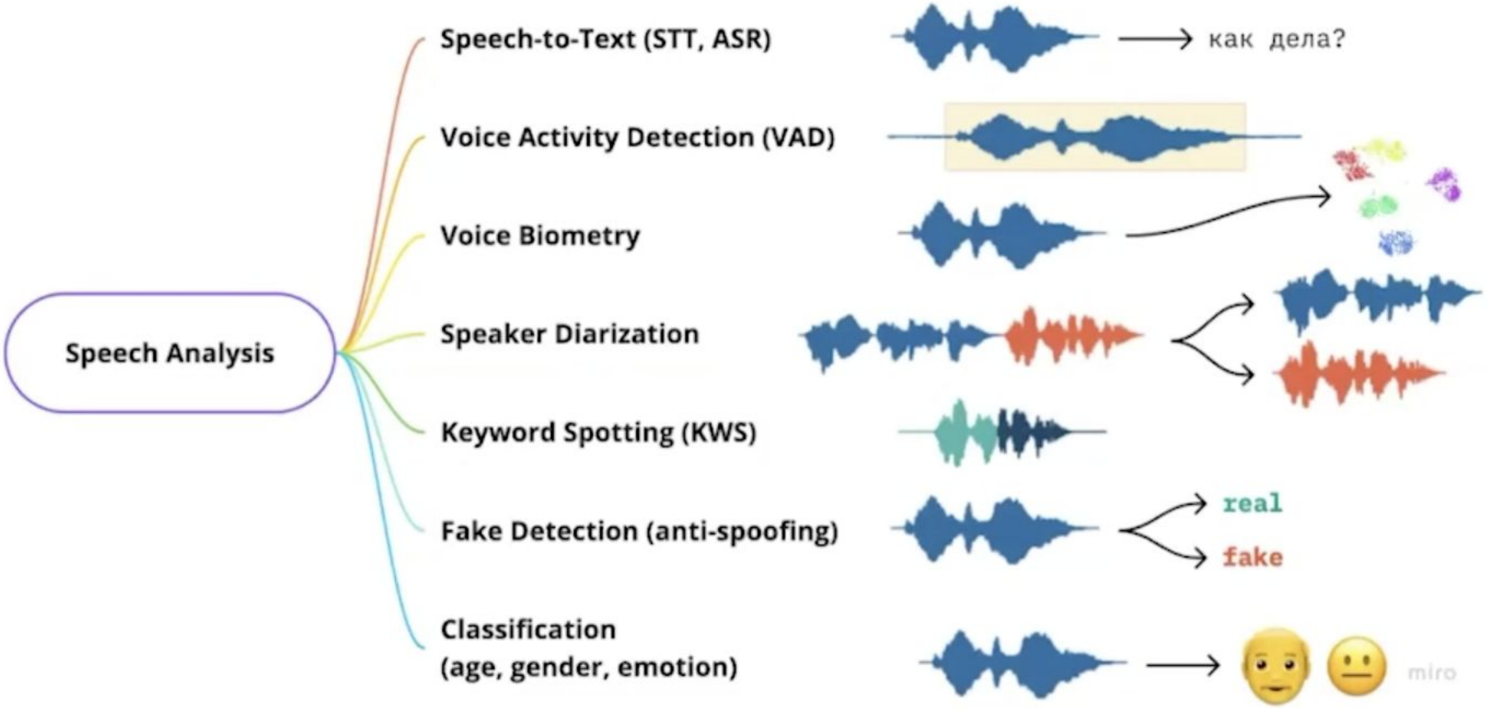
https://hugofloresgarcia.art/sketch2sound/

Voice Technologies Mind Map



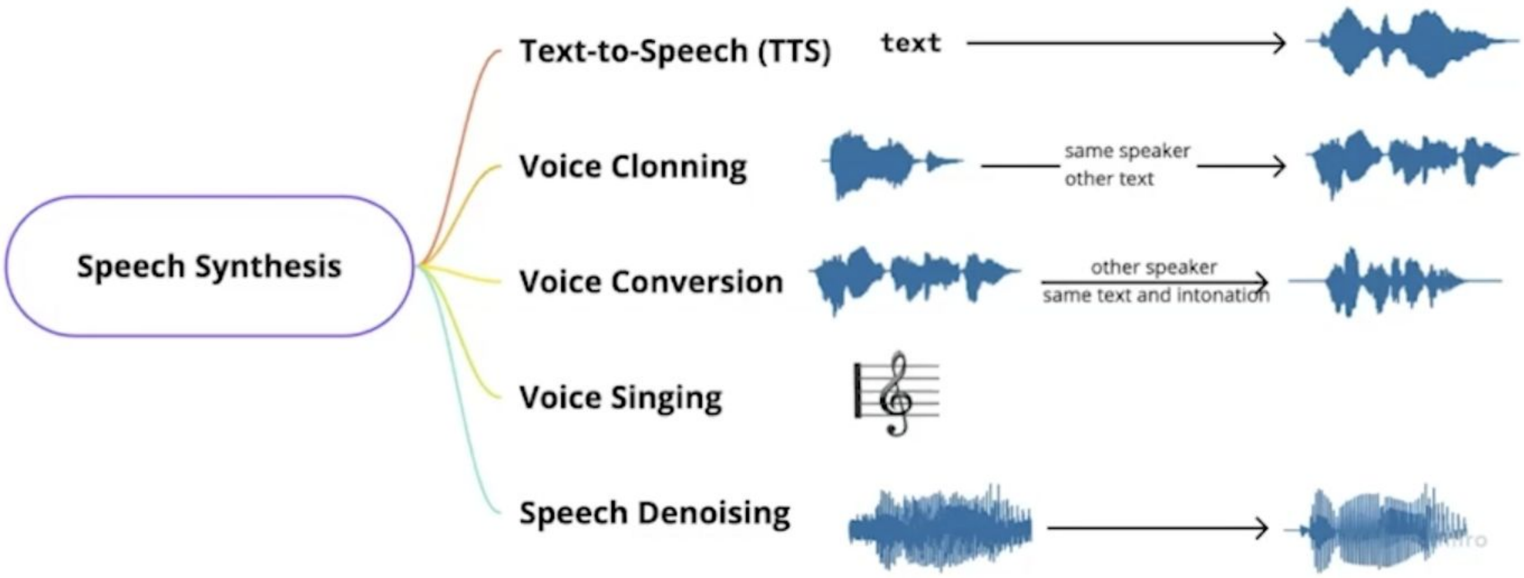
https://github.com/yandexdataschool/speech_course

Voice Technologies Mind Map



https://github.com/yandexdataschool/speech_course

Voice Technologies Mind Map



https://github.com/yandexdataschool/speech_course

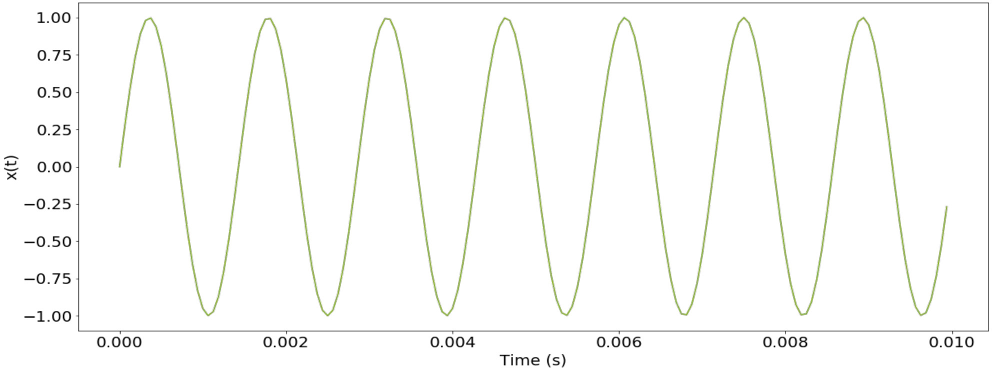
Brief Background for Core Assistant Parts

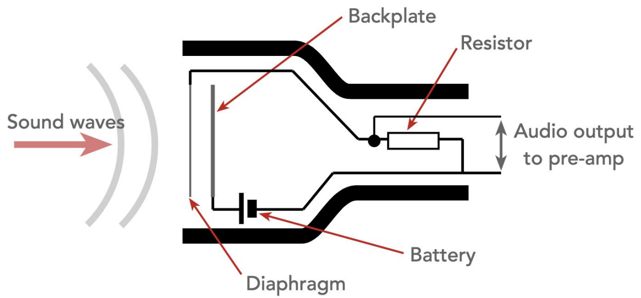
What is sound?

- **Sound wave** is the pattern of **oscillations** caused by the movement of energy traveling through the air

- **Microphone** picks up these air **oscillations** and converts them into electrical vibrations

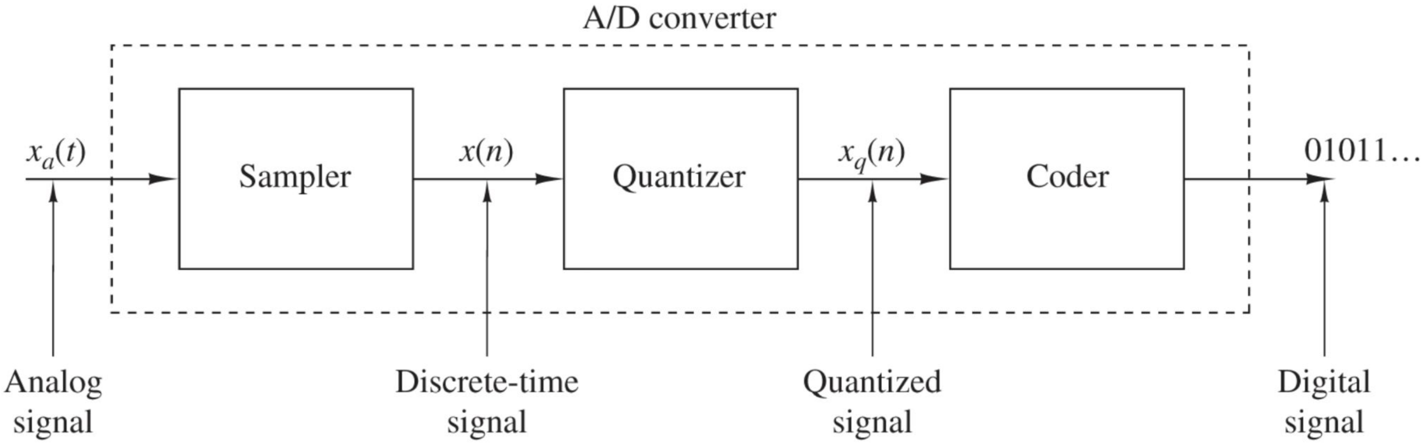
- These **oscillations** are converted into an **analog** signal and then into a **digital** signal





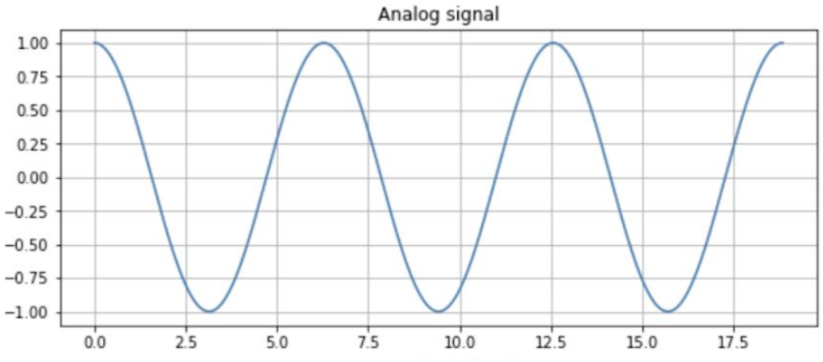
Analog-to-Digital Conversion

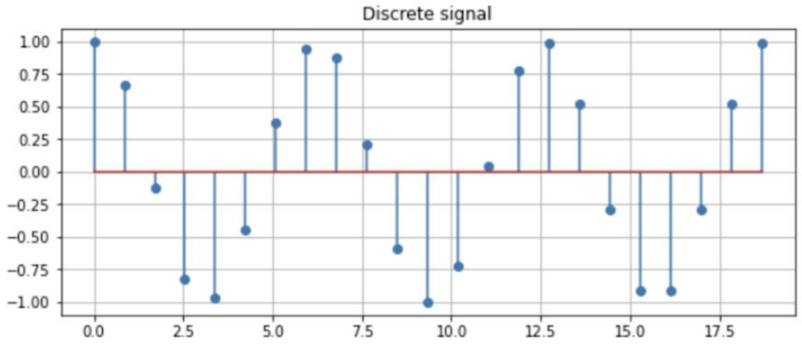
- Converting analog signals to a sequence of numbers having finite precision
- Corresponding devices are called A/D converters (ADCs)



Analog-to-Digital Conversion

- $X[n] = X(nT)$, where T is the **sampling period** in seconds
- **Sampling rate** (Hz) $f = 1 / T$, the number of audio samples in 1 sec.

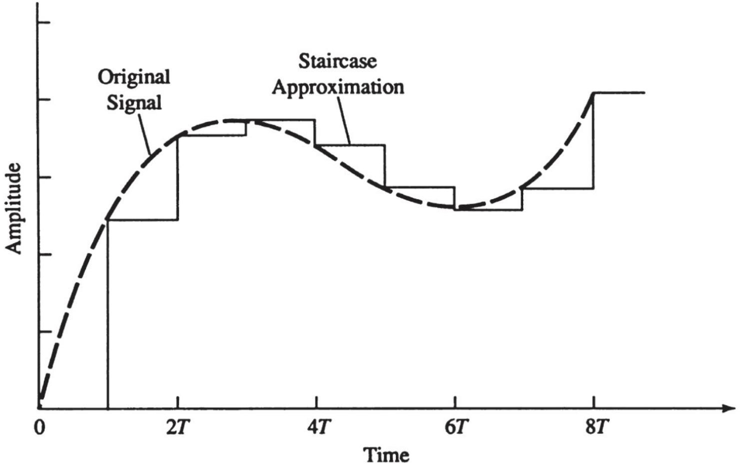




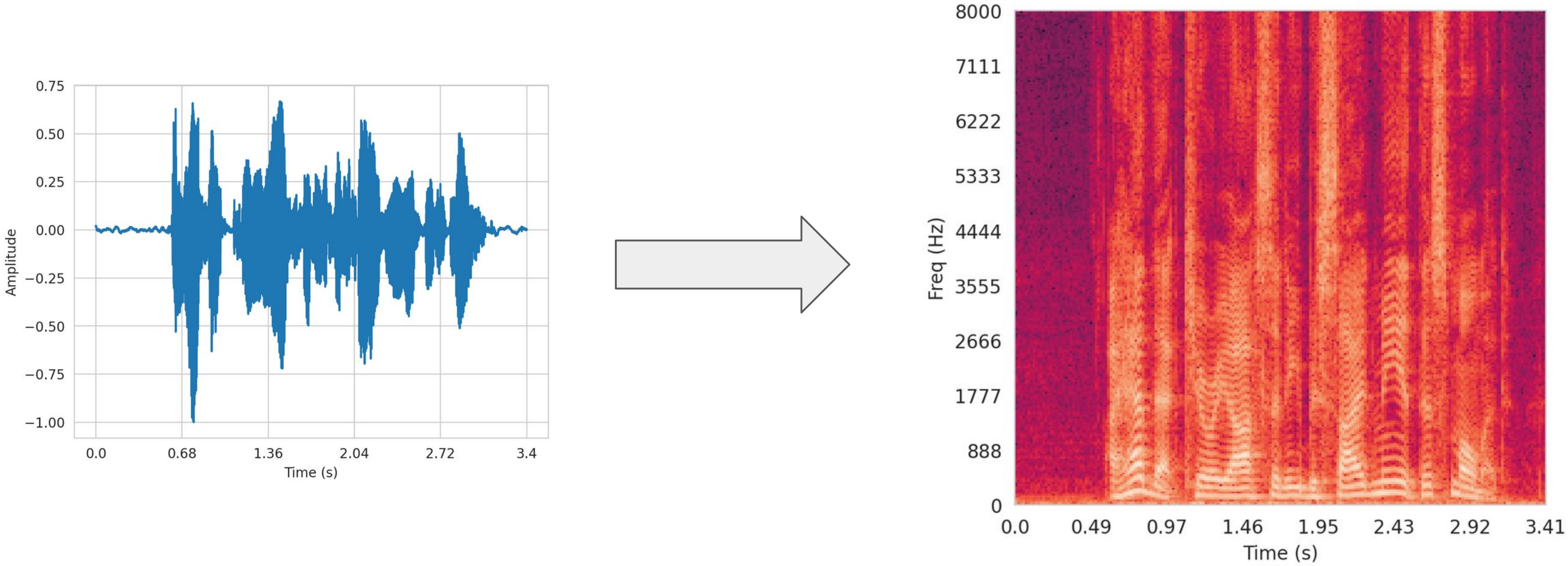
Usually Models are trained on recordings with a specific Sample rate. You should remember it and Resample audio before feeding it to the network.

Analog-to-Digital Conversion

- Using some interpolation techniques, we can convert digital signal back to analog one



Spectrograms



Problems with Raw Audio (Waveform) representation

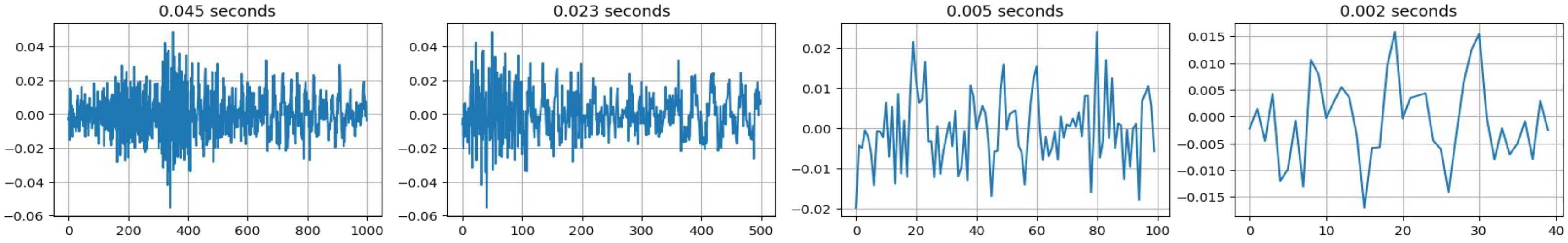
1 second of audio contains Sample Rate elements in it. This leads to severe computational costs

- For example, 4 minutes of audio with 44.1kHz sample rate results in a 10M-length sequence.

- In contrast, high resolution 1024x1024 image will require only 3M elements (in RGB)

Problems with Raw Audio (Waveform) representation

One letter (sound) requires thousands of samples (example for 22050 Hz sample rate):



Visualization is similar after transforms (like noise or pitch shift, etc) – hard to say anything about transform from the visualization

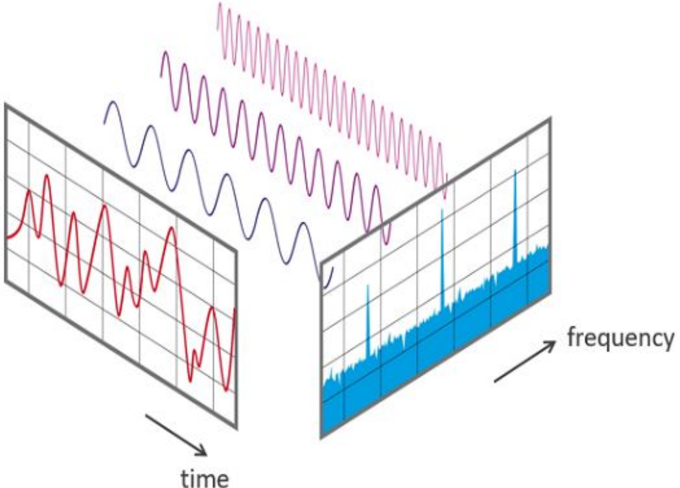
Voice

Voice + Noise

Frequencies

We need something apart from **temporal** representation to better understand what happens inside the audio, how it was modified, etc.

Frequency domain – another domain to represent audio whether some things may become clearer



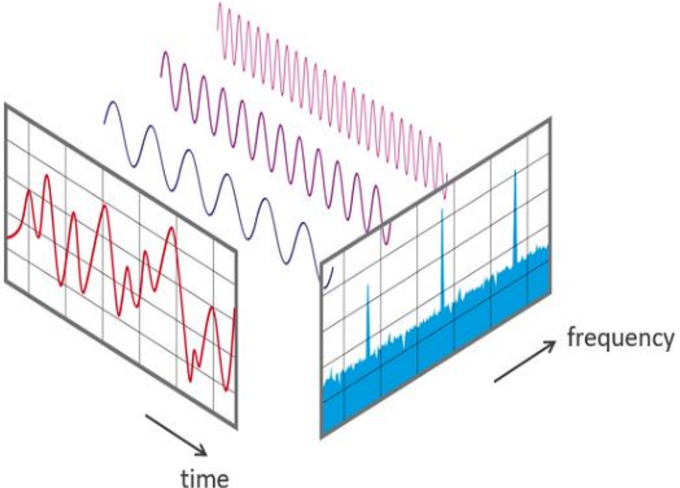
Frequencies – Fourier Transform

Sounds can be modeled as sum of sinusoid
→ determine frequencies and amplitudes

To find these sinusoids, we apply **Fourier Transform**, which is basically a **change of basis** in the vector space.

Fourier transform integral

$$\hat{f}(\xi) = \int_{-\infty}^{\infty} f(x)\, e^{-i2\pi\xi x}\, dx, \quad \forall\, \xi \in \mathbb{R}.$$



Frequencies – Fourier Transform

Let’s look at the visualization.

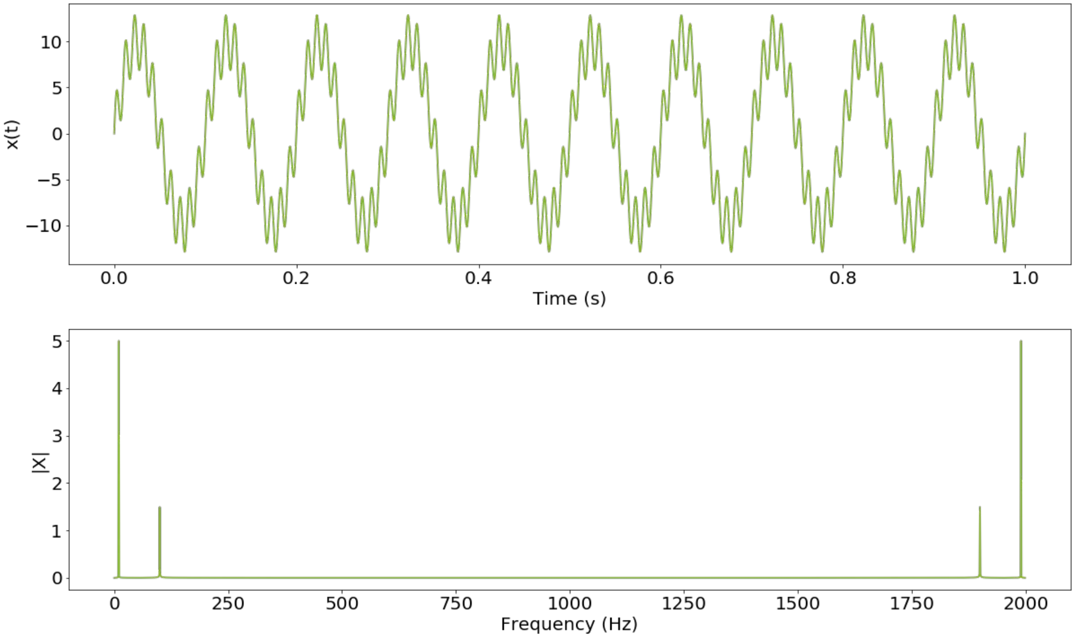
Remark 1: the output of fourier transform is complex (not real) by design.

Remark 2: In audio DL, we usually use discrete version

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{-i2\pi \frac{k}{N} n}$$

Frequencies – Fourier Transform

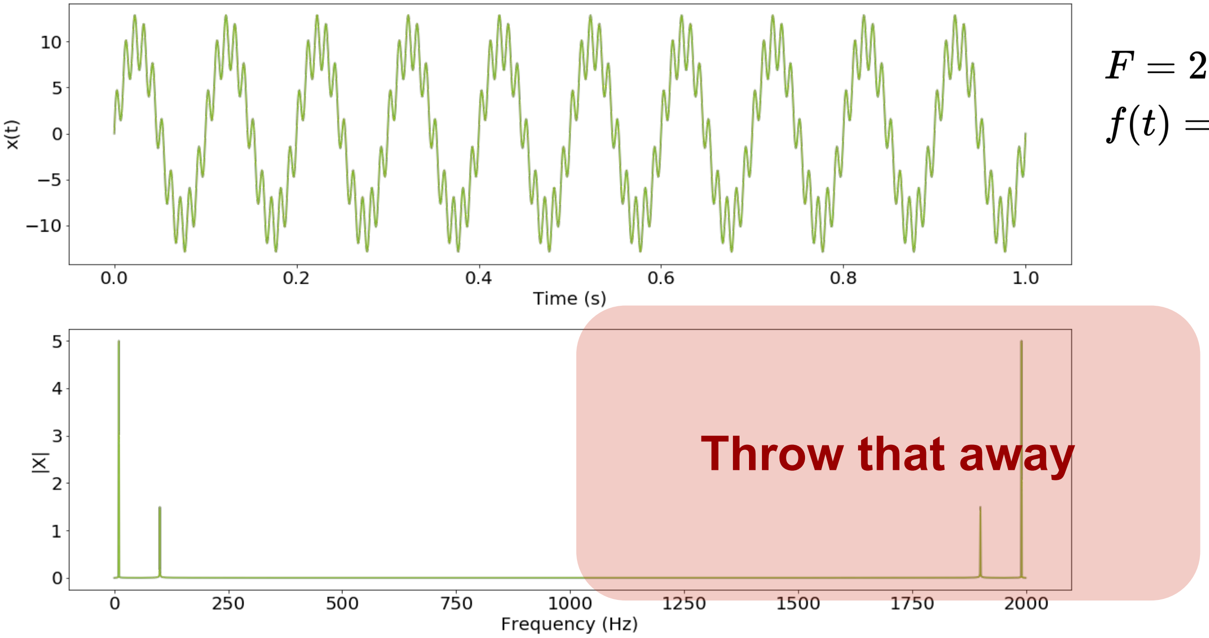
Note that Spectrum is symmetric for real inputs



$$F = 2kHz$$
$$f(t) = 10\sin(2\pi 10t) + 3\sin(2\pi 100t)$$

Frequencies – Fourier Transform

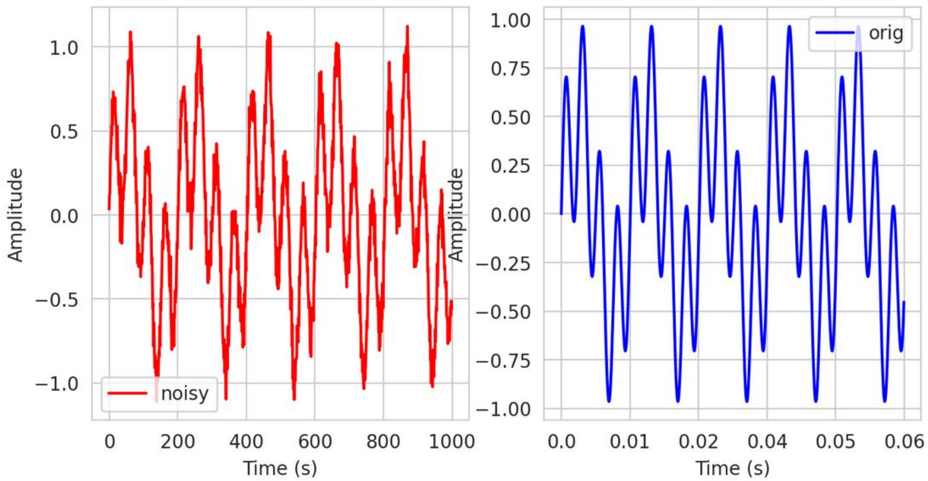
That’s why we usually work with only half of it

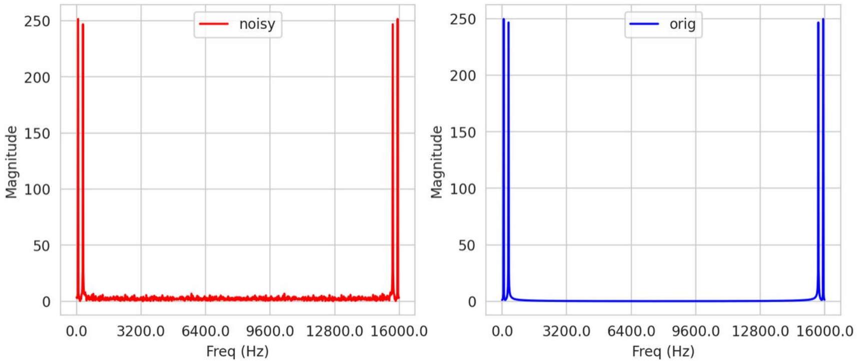


$$F = 2kHz$$
$$f(t) = 10\sin(2\pi 10t) + 3\sin(2\pi 100t)$$

Frequencies – Fourier Transform

Some stuff, like noise, may be easier to distinguish in the frequency domain:

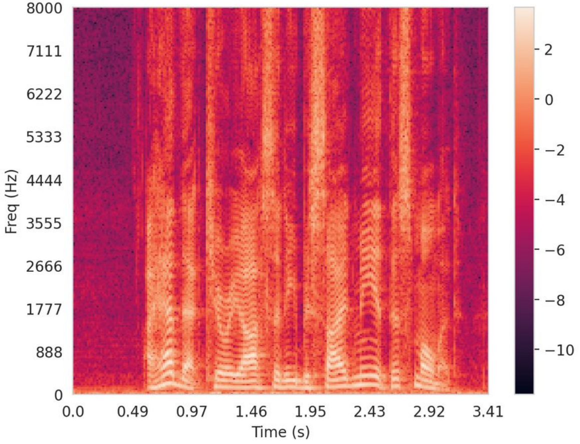




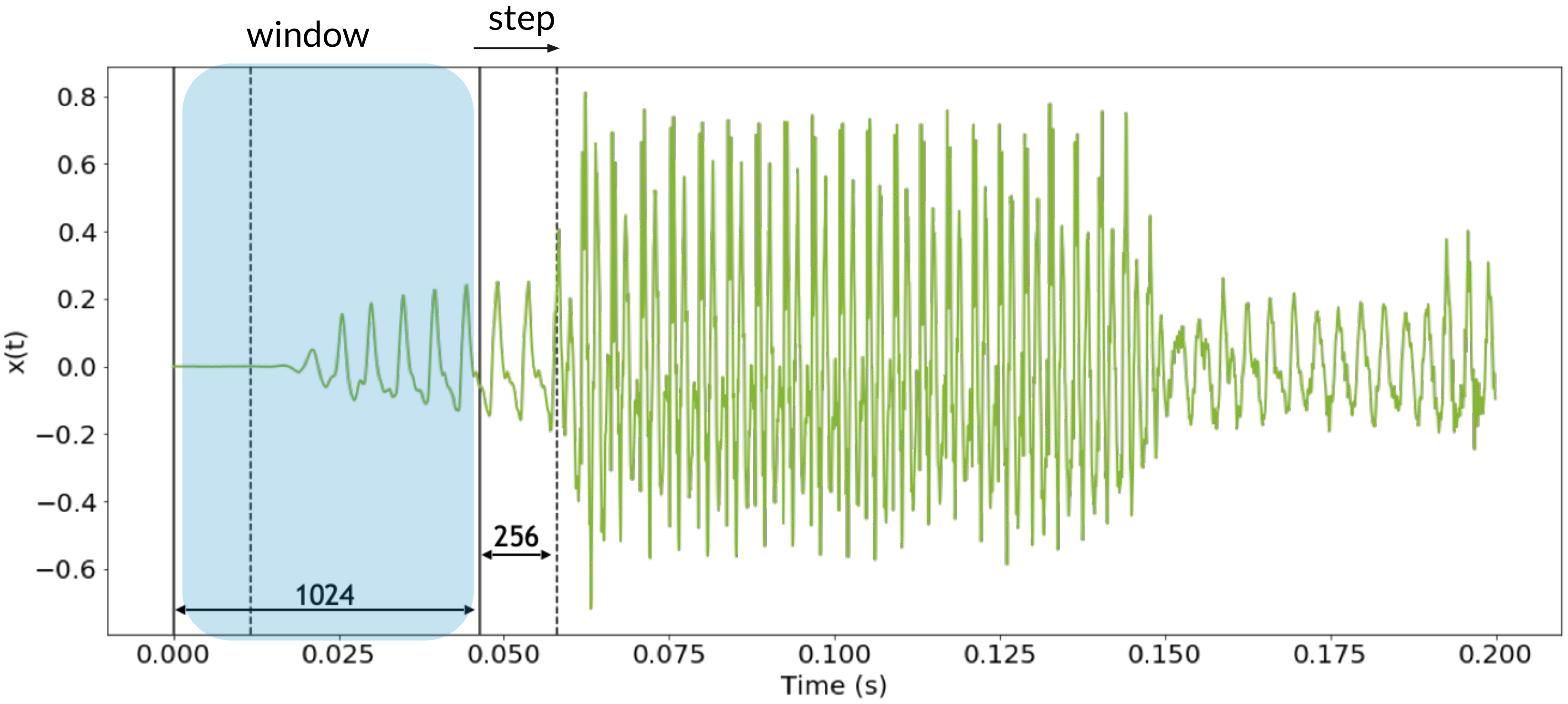
Time-Frequency Domain

Frequencies may rapidly change through time. So we want to see these changes

That's why, in practice, we work in Time-Frequency domain, where we see both changes across time and changes across frequencies



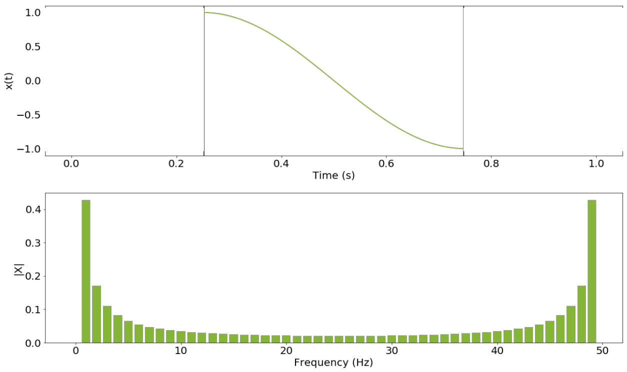
Short-Time Fourier Transform



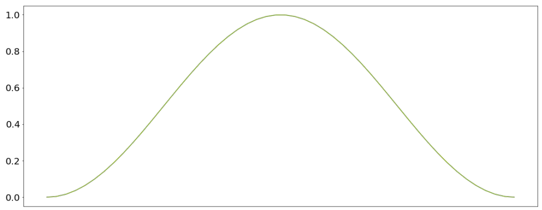
Short-Time Fourier Transform

Due to overlapping, we use window function to smooth the boundaries

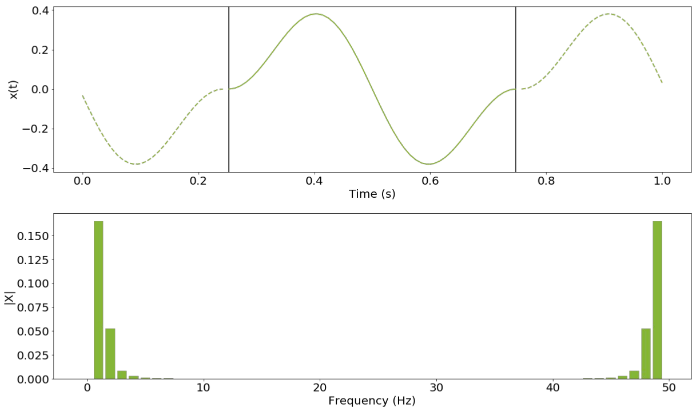
Sliced signal



Window

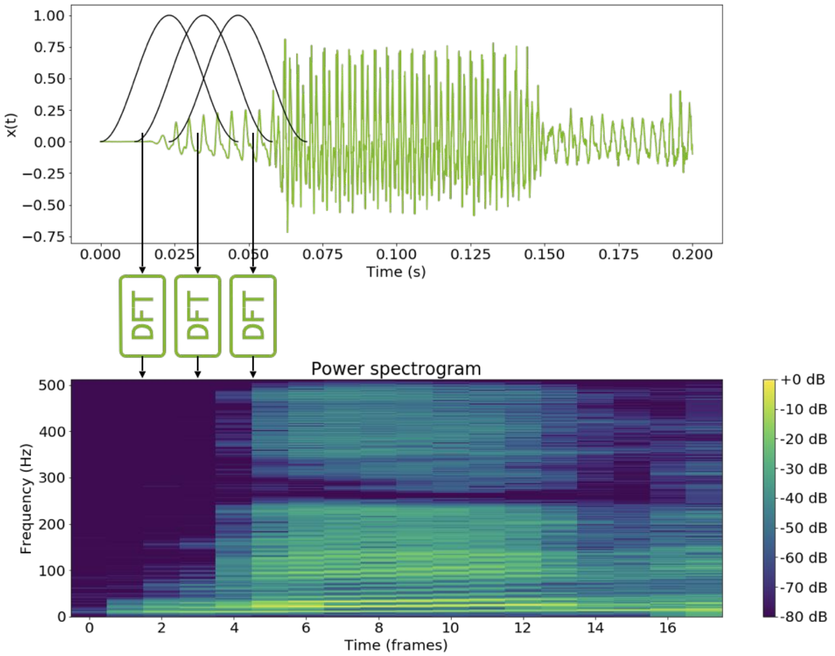


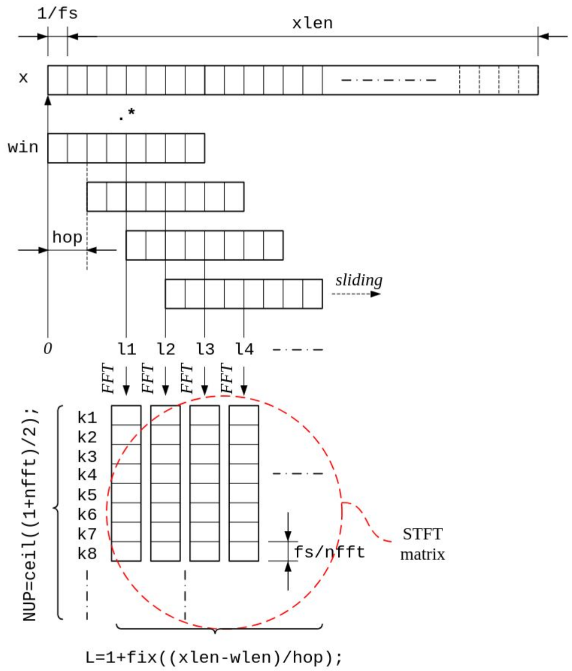
Windowed signal



Short-Time Fourier Transform

So we apply DFT for each window and stack the outputs

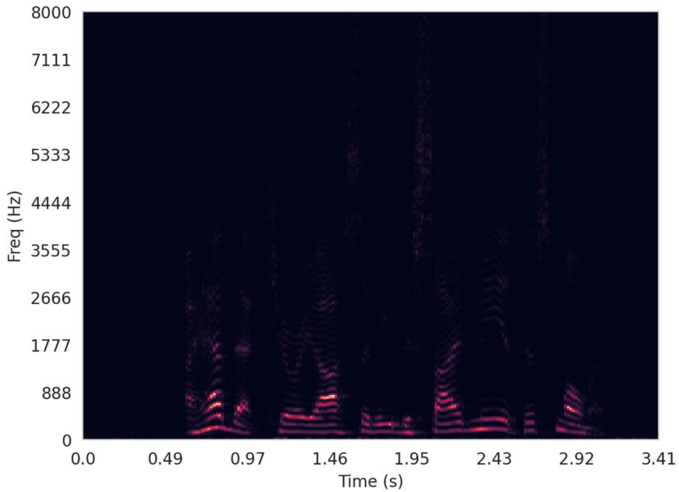


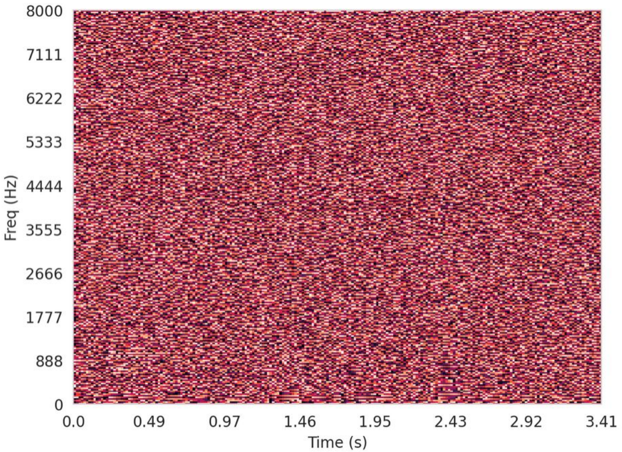


Spectrogram

Notice that the output of STFT is **complex-valued**. Usually we look at magnitude and phase of the output.

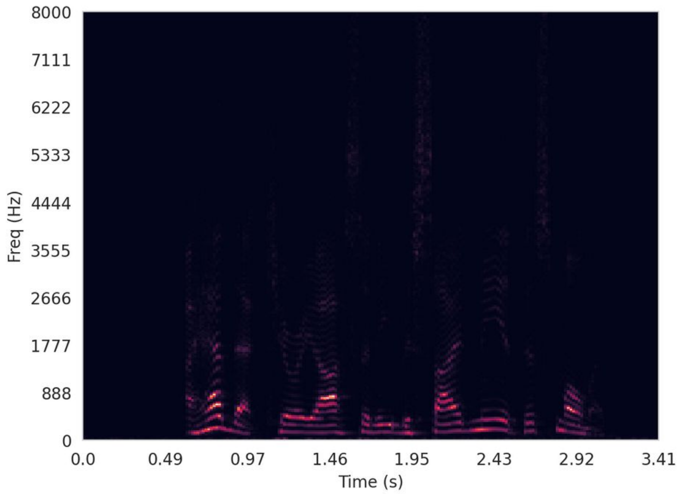
The magnitude version is called **Magnitude Spectrogram or just Spectrogram**. The phase version is called **Phase Spectrogram**.



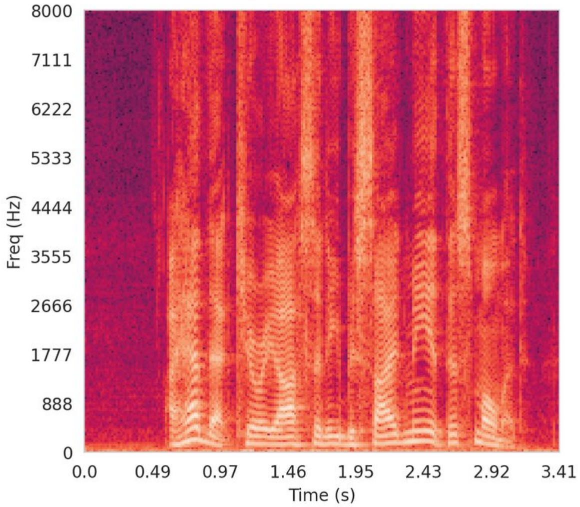


Spectrogram

Also, we often calculate the logarithm of the spectrogram, because it is much easier to interpret

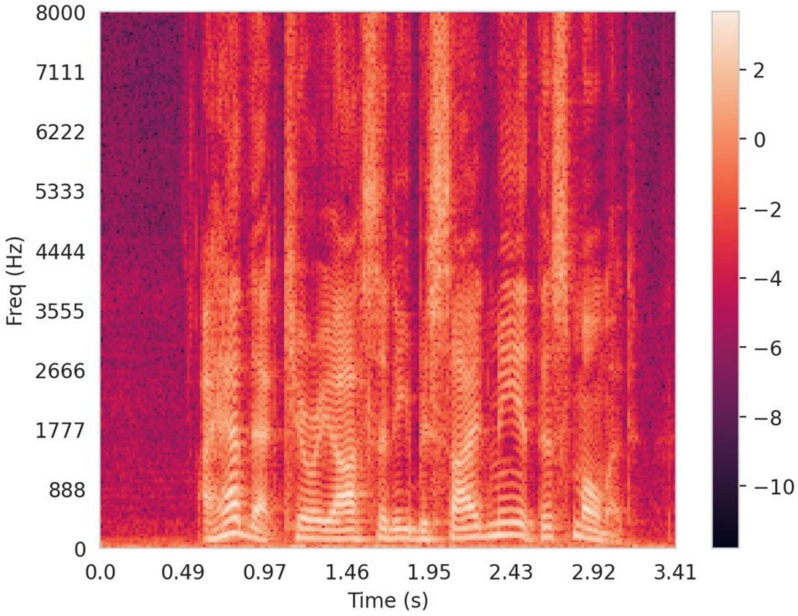


log

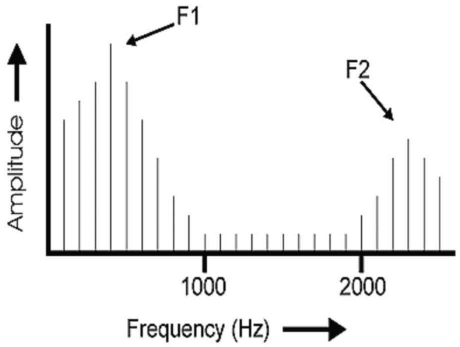


Short-Time Fourier Transform

STFT also allows us to see noise, etc. The speech signal, for example, will contain harmonics which can be clearly seen in the STFT



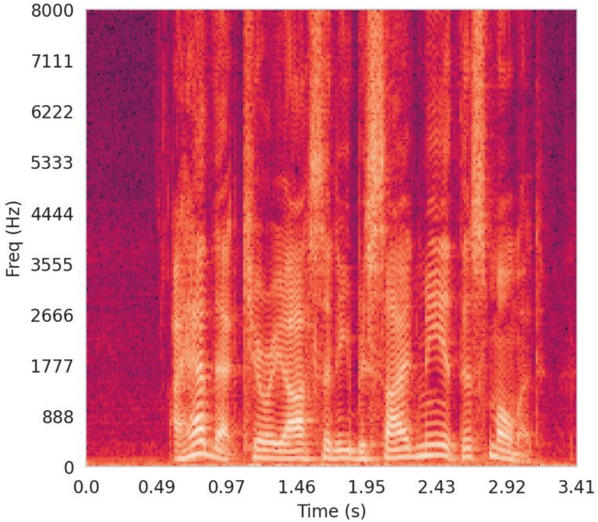
By looking at harmonics it is possible to detect what letter was said in this period!

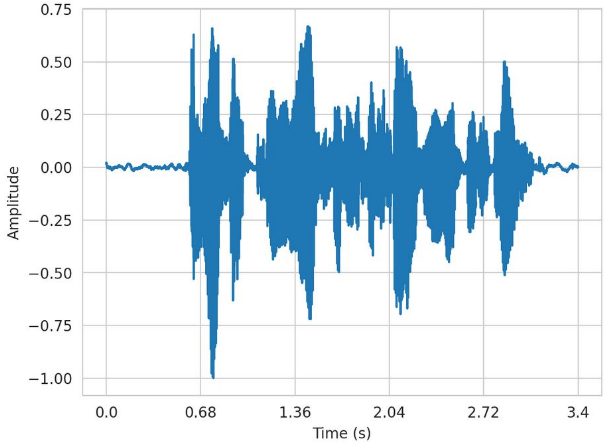


Remark: Inverse Transform

Fourier Transform is a change of basis. So it is invertible transform.

Given some constraints on window and hop (step) size, the STFT transform will also be **lossless** and we can invert to get original raw audio.

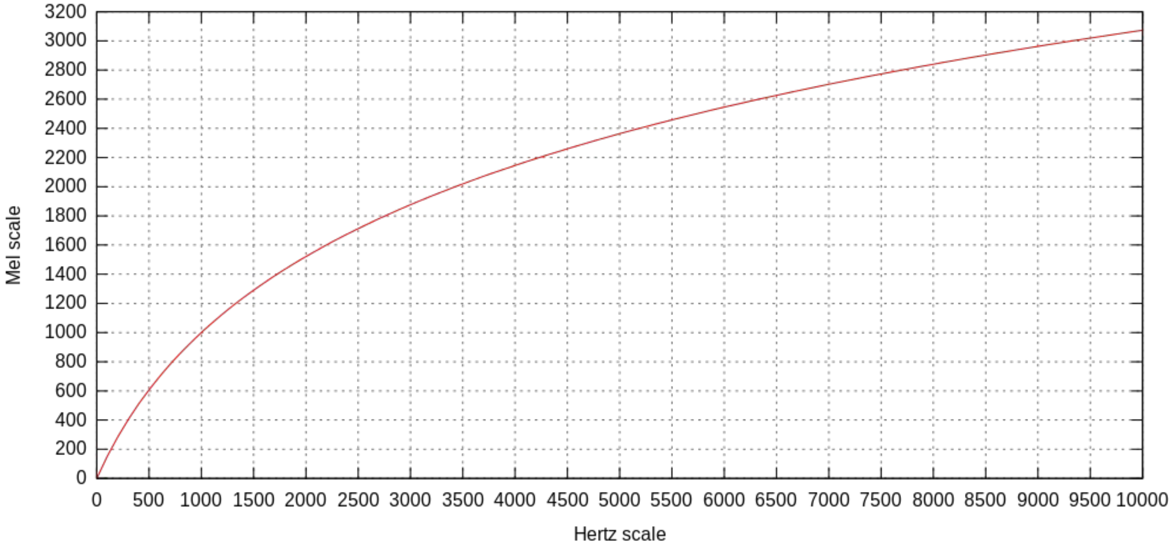




Mel Scale

Humans perceive sound on a log-scale. For human ear:

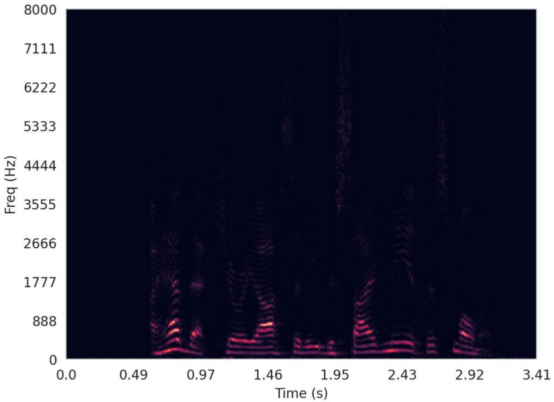
- 500 Hz << 600 Hz
- but 5000 Hz ~= 5100 Hz

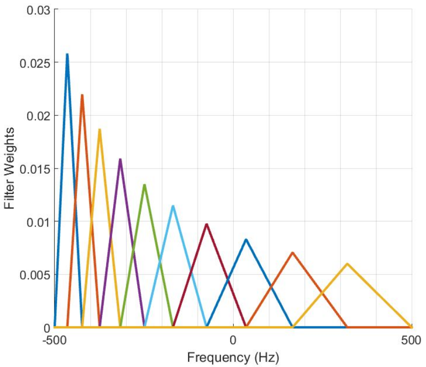


Mel Spectrogram

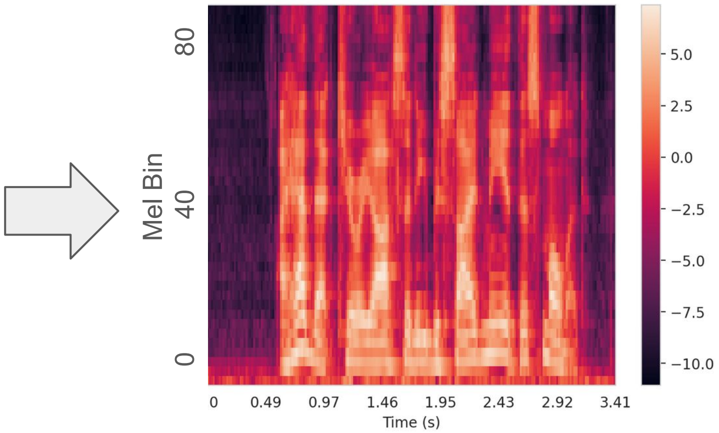
Many deep learning models use STFT in mel-scale instead. Note that this is a **lossy transform**

Weighted split on buckets





After log

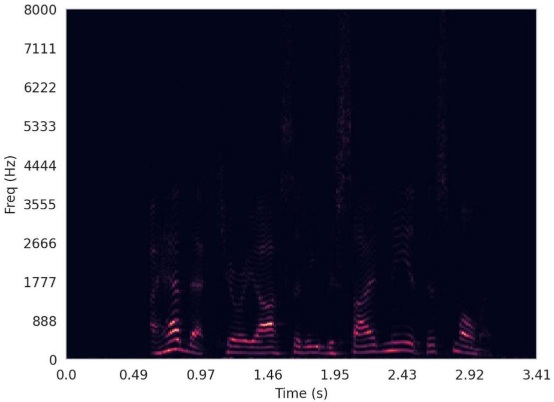


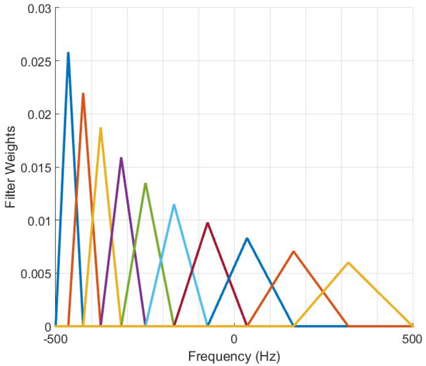
Implementation (librosa): https://librosa.org/doc/latest/generated/librosa.feature.melspectrogram.html#librosa.feature.melspectrogram

Mel Spectrogram

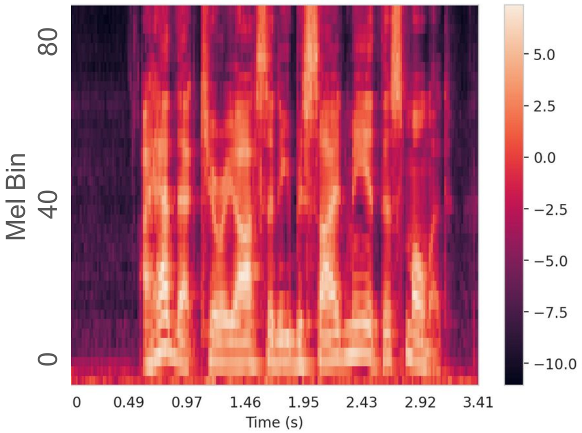
Remark: MelSpec is about making the **Y-axis look in log-scale**. Not about taking the log of the elements in the spectrogram!

Weighted split on buckets



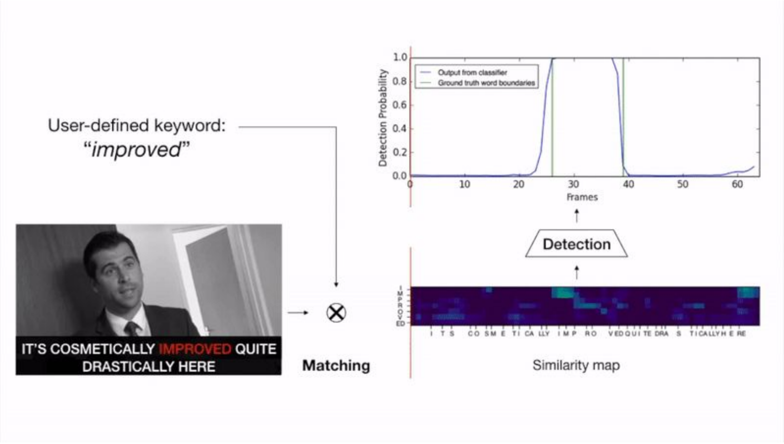


After log



Tasks Overview and Voice Assistant

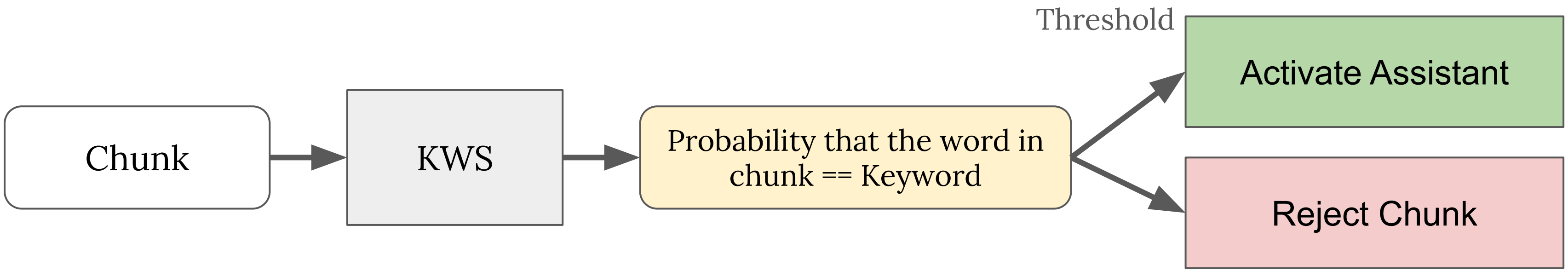
KWS



Momeni, Liliane, et al. "Seeing wake words: Audio-visual keyword spotting." arXiv preprint arXiv:2009.01225 (2020).

KWS is just a classification task.

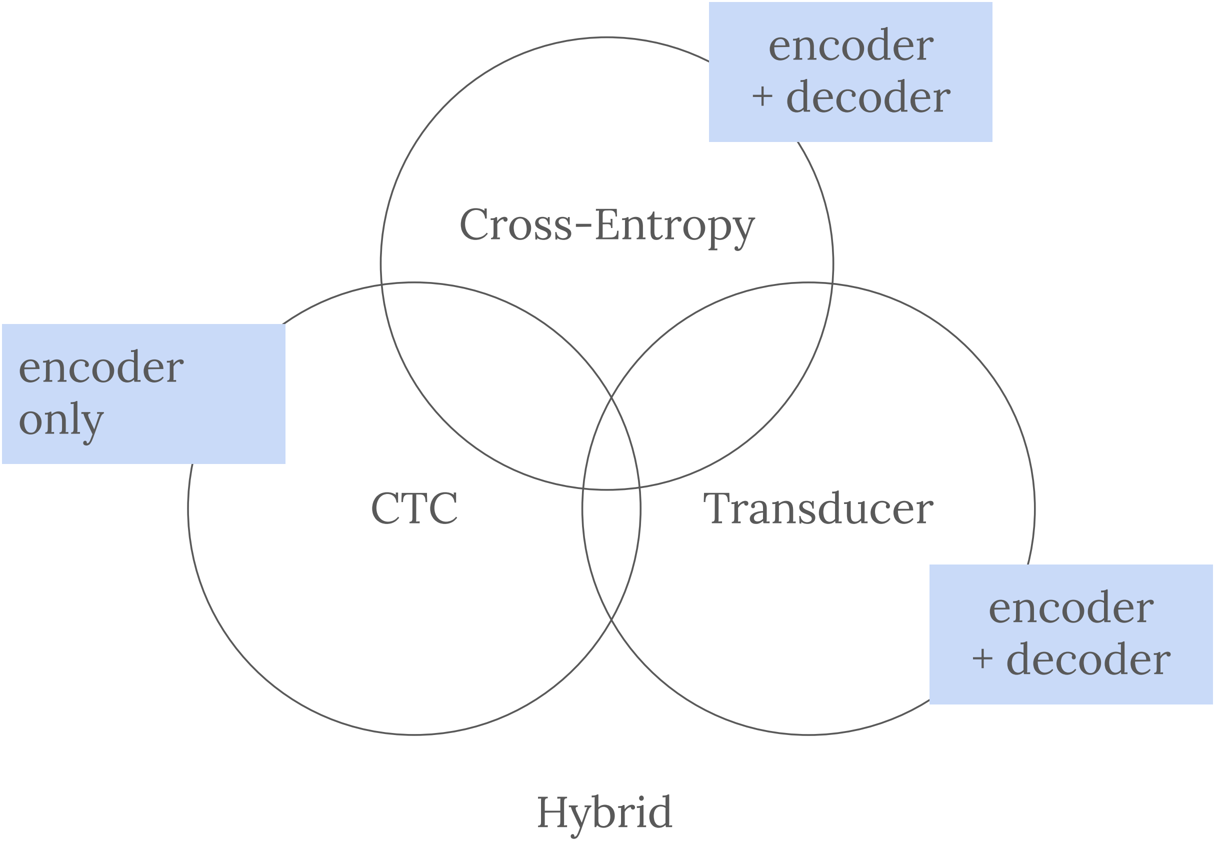
Given a short signal with only one word, model is trained to distinguish keyword (label 1) from any other word (label 0)



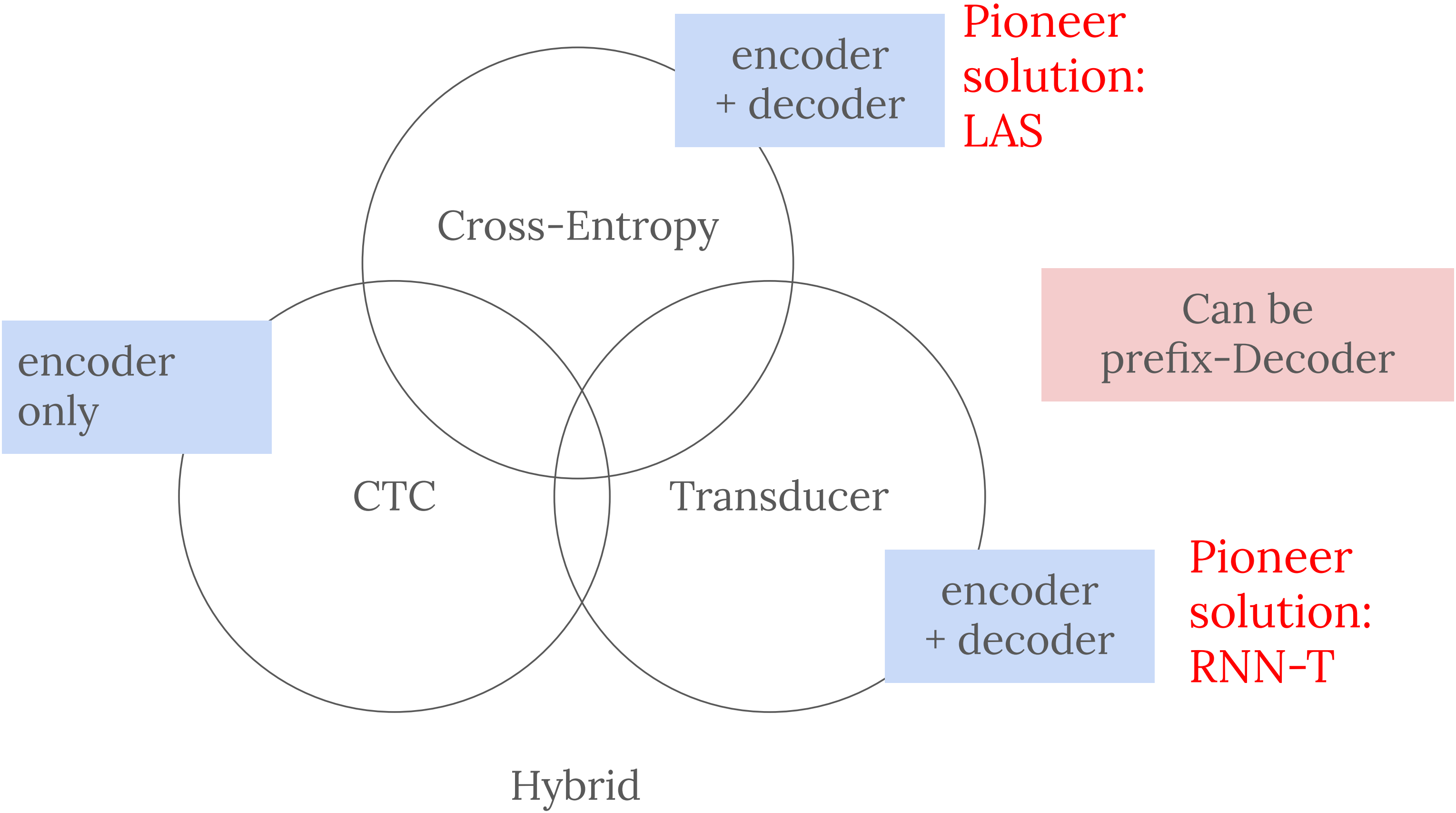
KWS

Let’s see how to train such a model in PyTorch

Types of ASR models

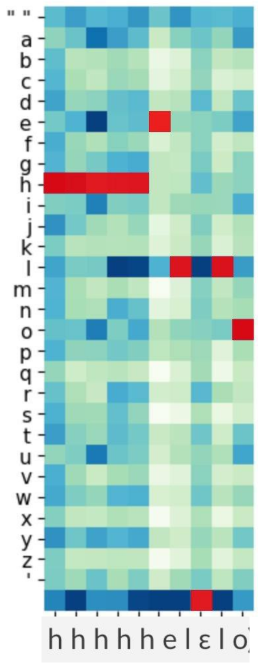


Types of ASR models



Automatic Speech Recognition (Speech To Text)

Why should you care? Each type has its own inference algorithms. Each algorithm also has its own sub-versions which may affect quality severely.

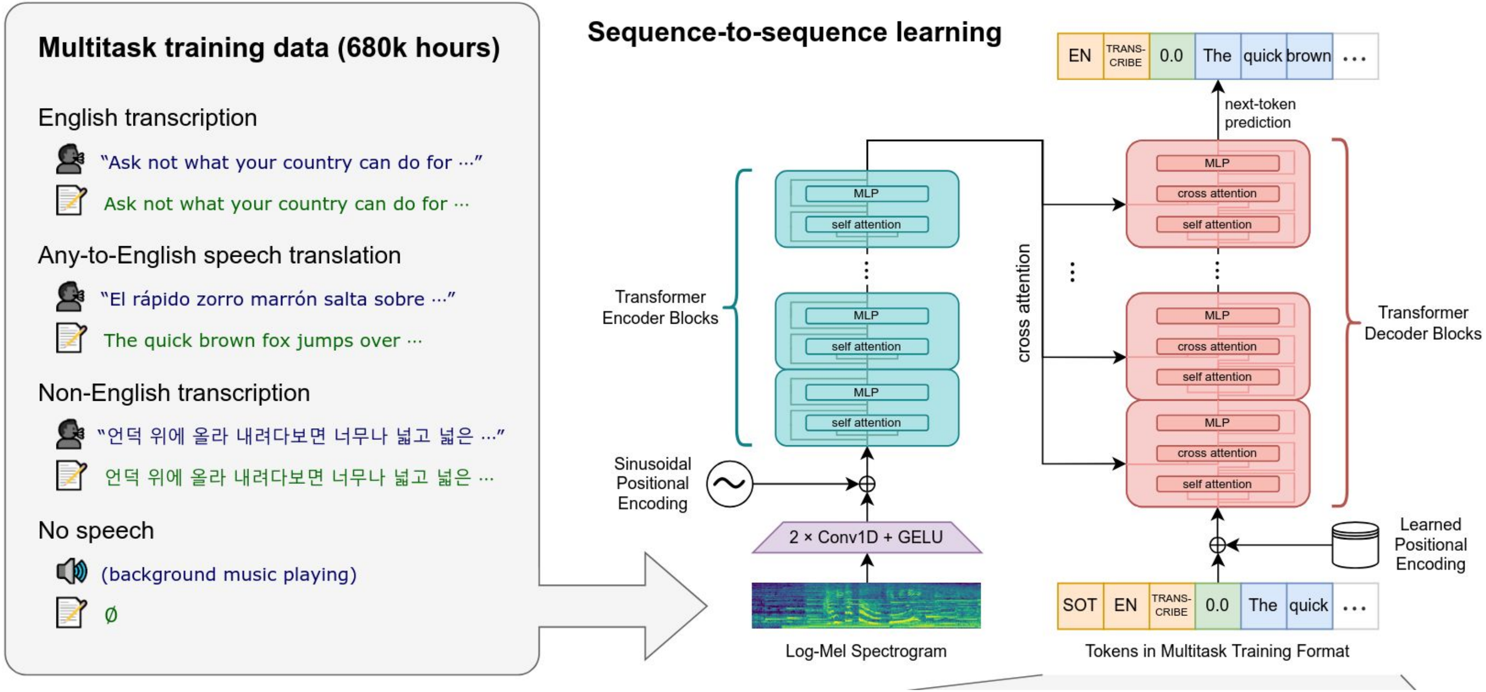


For example, in CTC, your model returns such probability matrix *time_step x vocabulary_size*

You can decode it into text via several ways:

- Greedy Search: Fast, but the quality suffer
- Beam Search: Slow, better quality
- Beam Search with Language Model: Even slower, remarkably better quality

Autoregressive Variant: Whisper

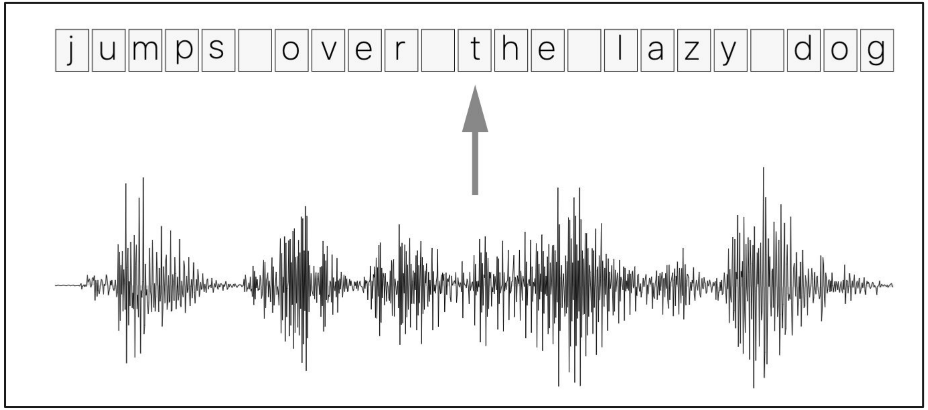


[Robust Speech Recognition via Large-Scale Weak Supervision (ICML 23)]

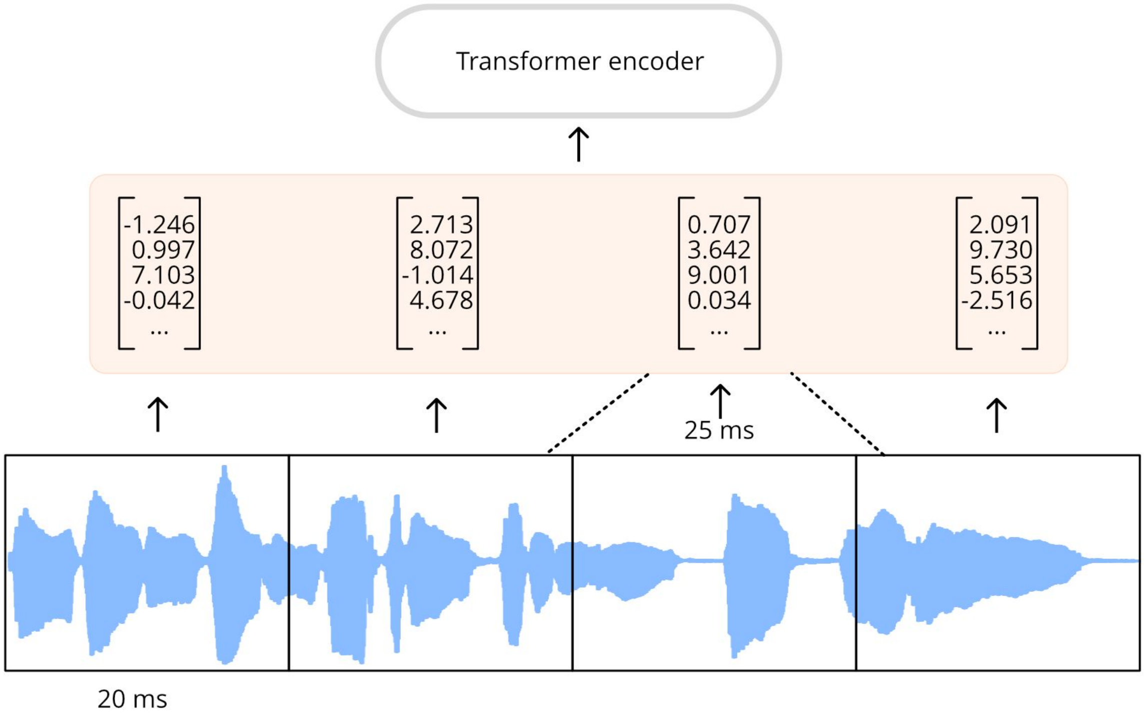
Connectionist Temporal Classification (CTC)

In CTC, you have a vocabulary of size V: all possible tokens (in a simple case, letters from the alphabet and space: “ “)

Given speech input, the model returns the tensor TxV. Where each row i represents the probability that corresponding token is said at timestep i.



Typical encoder



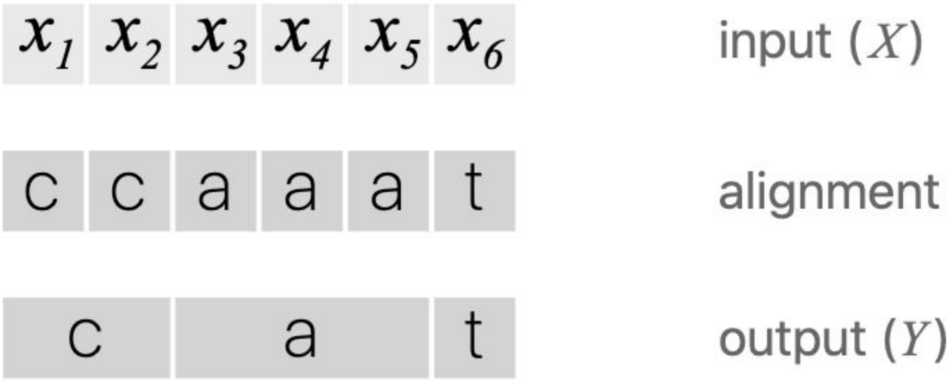
- Input audio is chunked (e.g. 20 ms)
- CTC outputs **one phoneme or character per chunk**

Image source: https://huggingface.co/learn/audio-course/chapter3/ctc

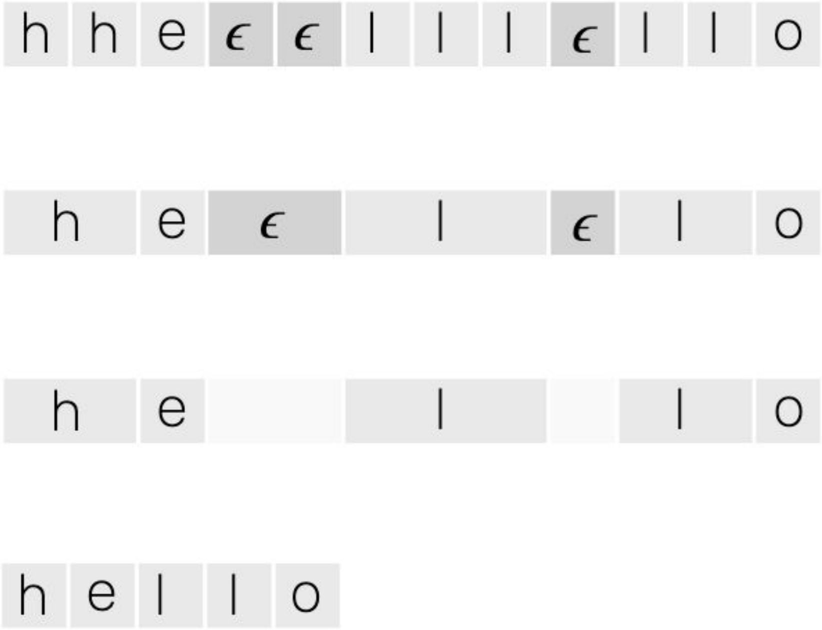
Connectionist Temporal Classification (CTC)

- Split input into frames
- Classify each frame into num_letters (|V|) classes
- Merge consecutive letters

Problem:
cannot distinguish some words:
He**ll**o vs Helo



Connectionist Temporal Classification (CTC)



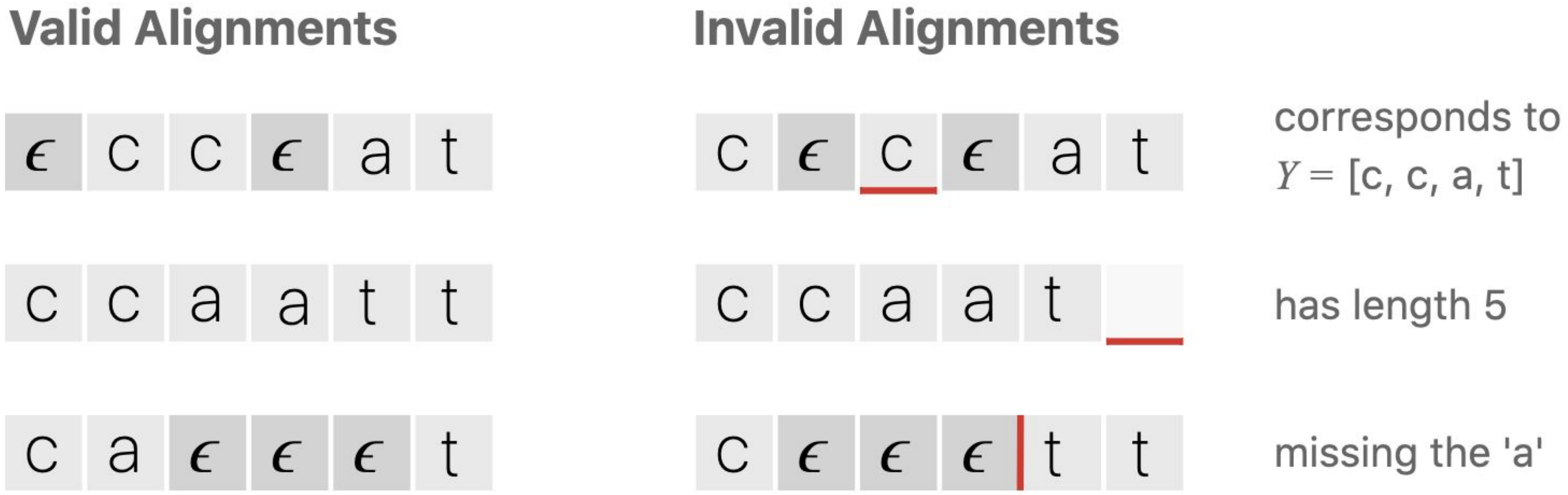
Solution: Add special token

First, merge repeat characters.

Then, remove any ϵ tokens.

The remaining characters are the output.

Connectionist Temporal Classification (CTC)



CTC Loss Function

We start with an input sequence, like a spectrogram of audio.

The input is fed into an RNN, for example.

The network gives $p_t(a \mid X)$, a distribution over the outputs {h, e, l, o, ϵ} for each input step.

With the per time-step output distribution, we compute the probability of different sequences

By marginalizing over alignments, we get a distribution over outputs.

CTC Loss Function

$$p(Y \mid X) \quad = \quad \sum_{A \in \mathcal{A}_{X,Y}} \quad \prod_{t=1}^{T} p_t(a_t \mid X)$$

The CTC conditional **probability**

marginalizes over the set of valid alignments

computing the **probability** for a single alignment step-by-step.

Can be calculated effectively via **dynamic programming** algorithm

$$\begin{aligned} p(\text{prefix} = \text{'cat'})_T \quad = \\ p(\text{prefix} = \text{'cat'}, \text{last\_char} = \text{'t'})_T \\ + \, p(\text{prefix} = \text{'cat'}, \text{last\_char} = \epsilon)_T \end{aligned}$$

$$\begin{aligned} p(\text{prefix} = \text{'cat'}, \text{last\_char} = \text{'t'})_T = \\ p(\text{prefix} = \text{'ca'}, \text{last\_char} = \text{'a'})_{T-1} \cdot p(\text{'t'})_T \\ + p(\text{prefix} = \text{'ca'}, \text{last\_char} = \epsilon)_{T-1} \cdot p(\text{'t'})_T \\ + p(\text{prefix} = \text{'ca} \;\;, \text{last\_char} = \text{'t'})_{T-1} \cdot p(\text{'t'})_T \end{aligned}$$

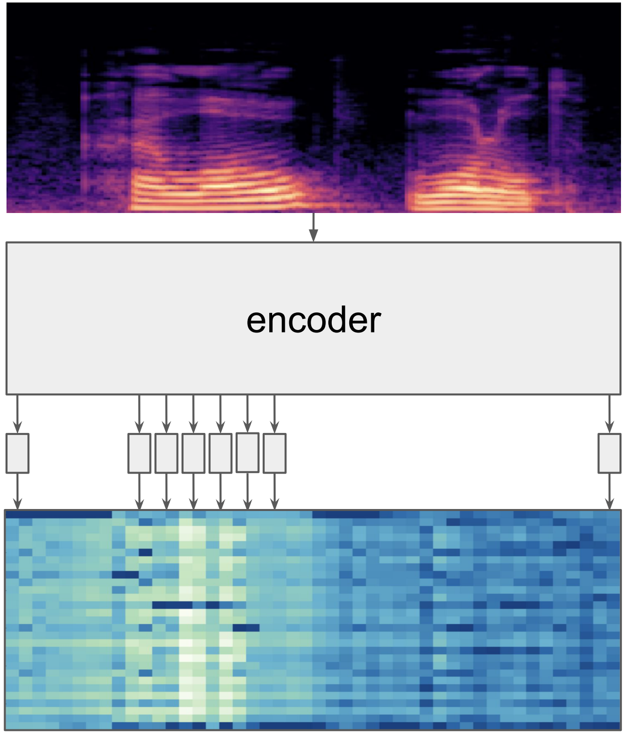
CTC Loss Function

In PyTorch, everything is simple:

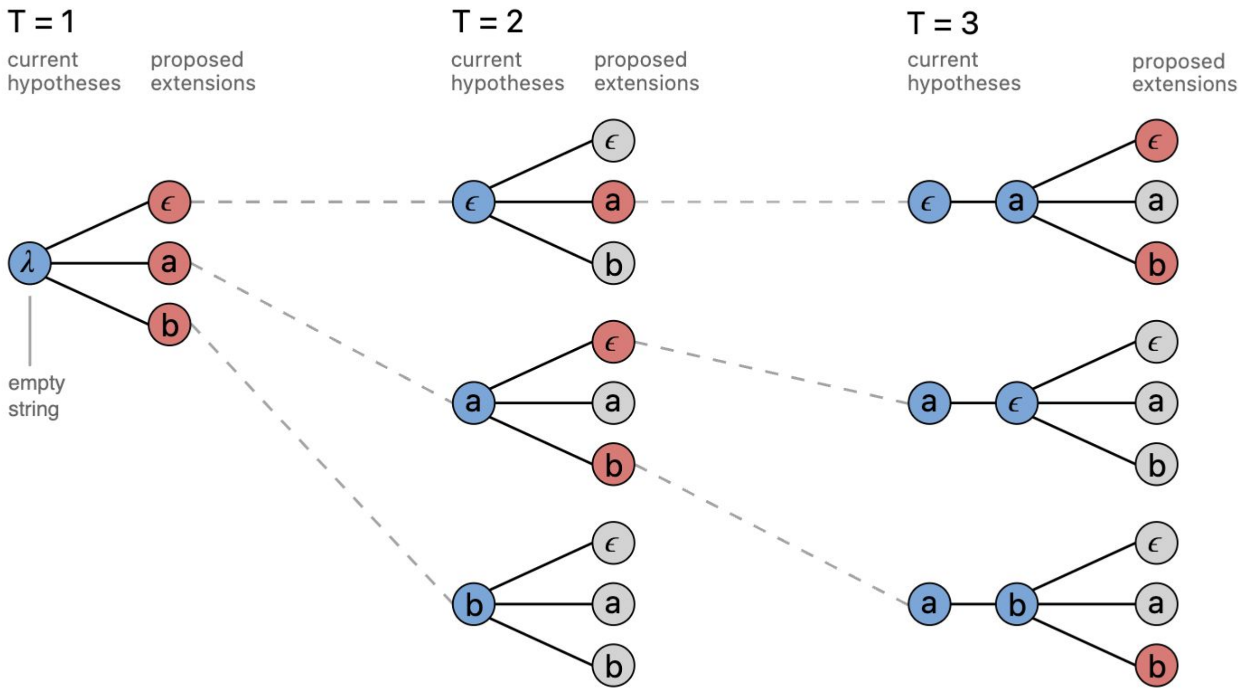
```
from torch.nn import CTCLoss

criterion = CTCLoss()
loss = criterion(
    log_probs=log_probs, # T x B x |V|
    targets=text_encoded, # B x S (length of tokenized text)
    input_lengths=log_probs_length,# B x T (length without padding)
    target_lengths=text_encoded_length, # B x S (length without padding)
)
```

CTC Beam Search

- Want to decode an output matrix into a list of text predictions with probabilities
- Has parameter: beam size >= 1
- Tradeoff between:
  - taking argmax prediction - easy and fast to compute, but not perfectly accurate
  - computing sum of probabilities of all possible paths - perfect quality, but infeasible



Beam Search

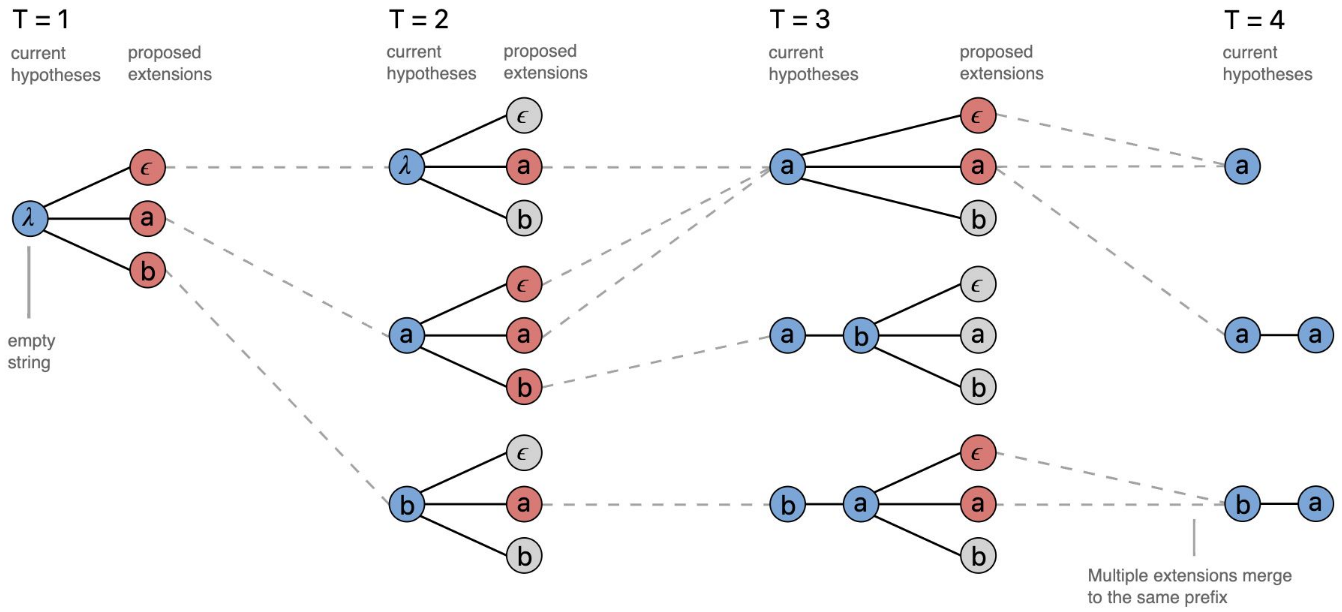


A standard beam search algorithm with an alphabet of $\{\epsilon, a, b\}$ and a beam size of three.

Beam search step:
- expand beam
- truncate beam

https://distill.pub/2017/ctc/

CTC Beam Search



The CTC beam search algorithm with an output alphabet $\{\epsilon, a, b\}$ and a beam size of three.

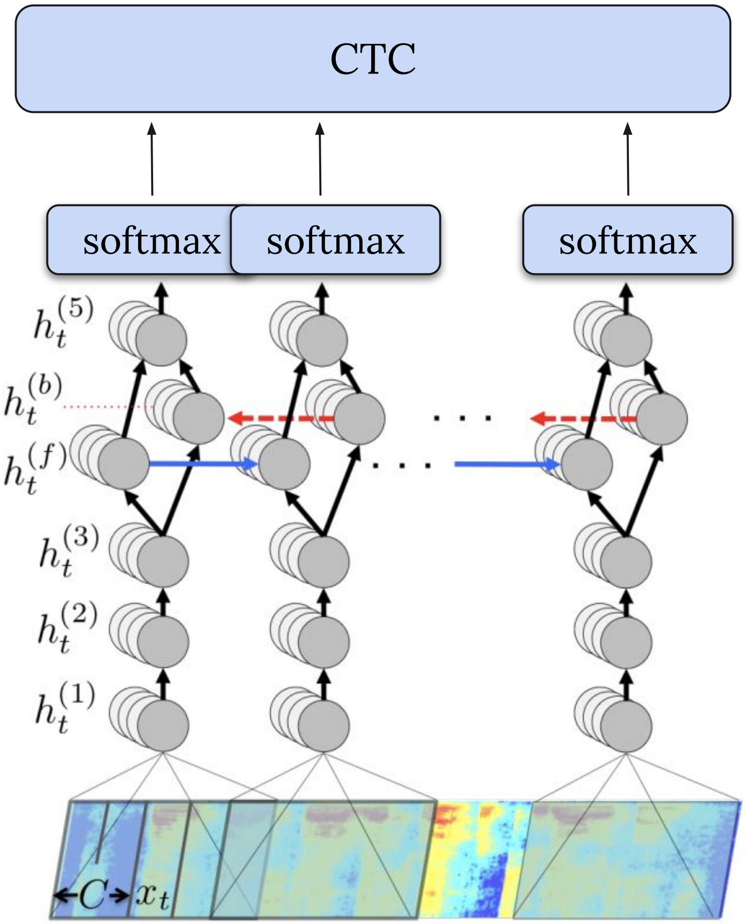
Beam search step:
- expand beam
- merge paths
- truncate beam

https://distill.pub/2017/ctc/

Deep Speech (2014)

Input: Spectrogram

Model: MLP + Bi-LSTM

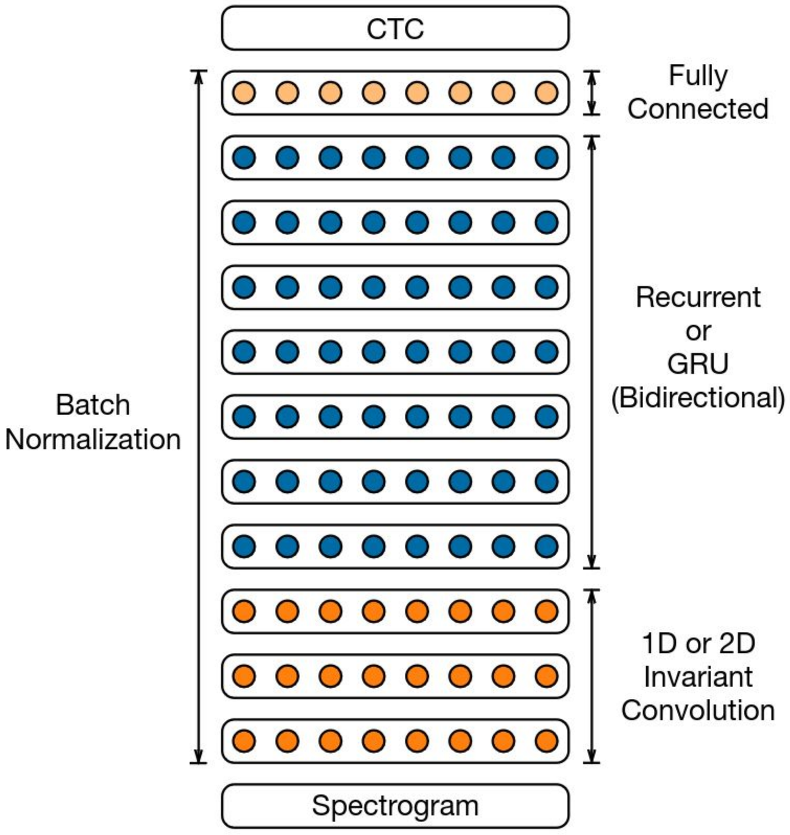


[Deep Speech: Scaling up end-to-end speech recognition (2014)](#)

Deep Speech 2 (2015)

Input: Spectrogram

Model: CNN + Bi-LSTM

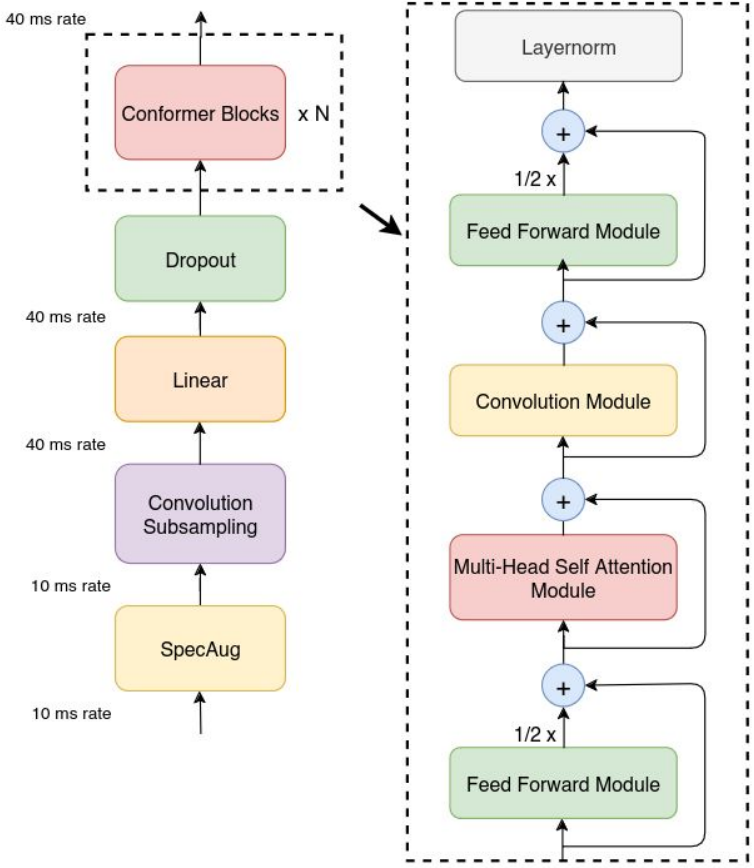


[Deep Speech 2: End-to-End Speech Recognition in English and Mandarin (2015)](#)

Conformer (2020)

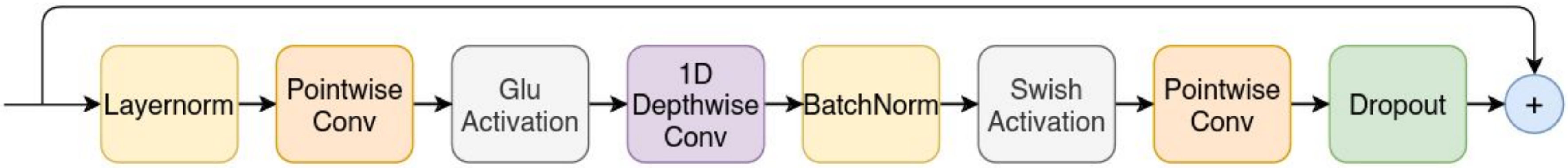
Input: Spectrogram

Model: Modified Transformer

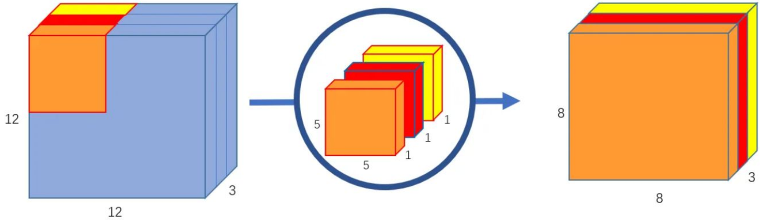


[Conformer: Convolution-augmented Transformer for Speech Recognition (Interspeech 2020)]

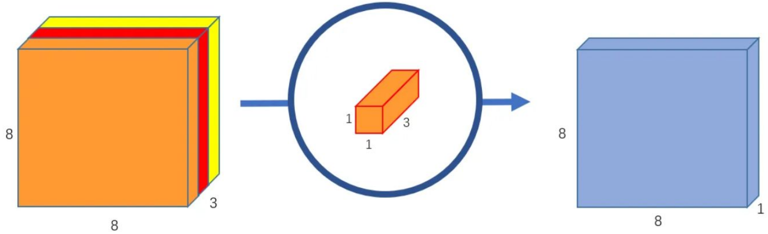
Conformer (2020) - Convolution Module



Depthwise Convolution



Pointwise Convolution

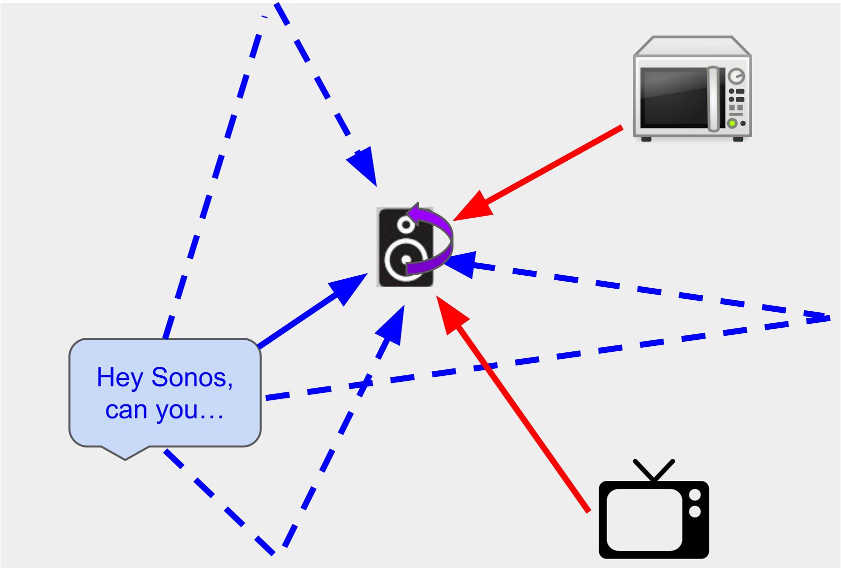


Far-field ASR

Reverberation

Interferences

Self-sound

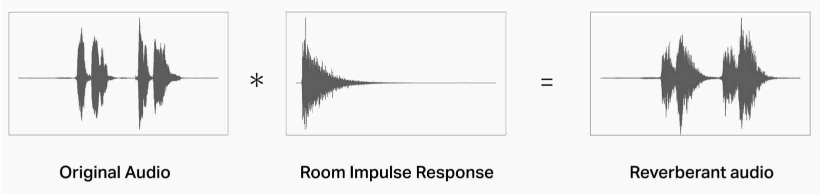


Dereverberation

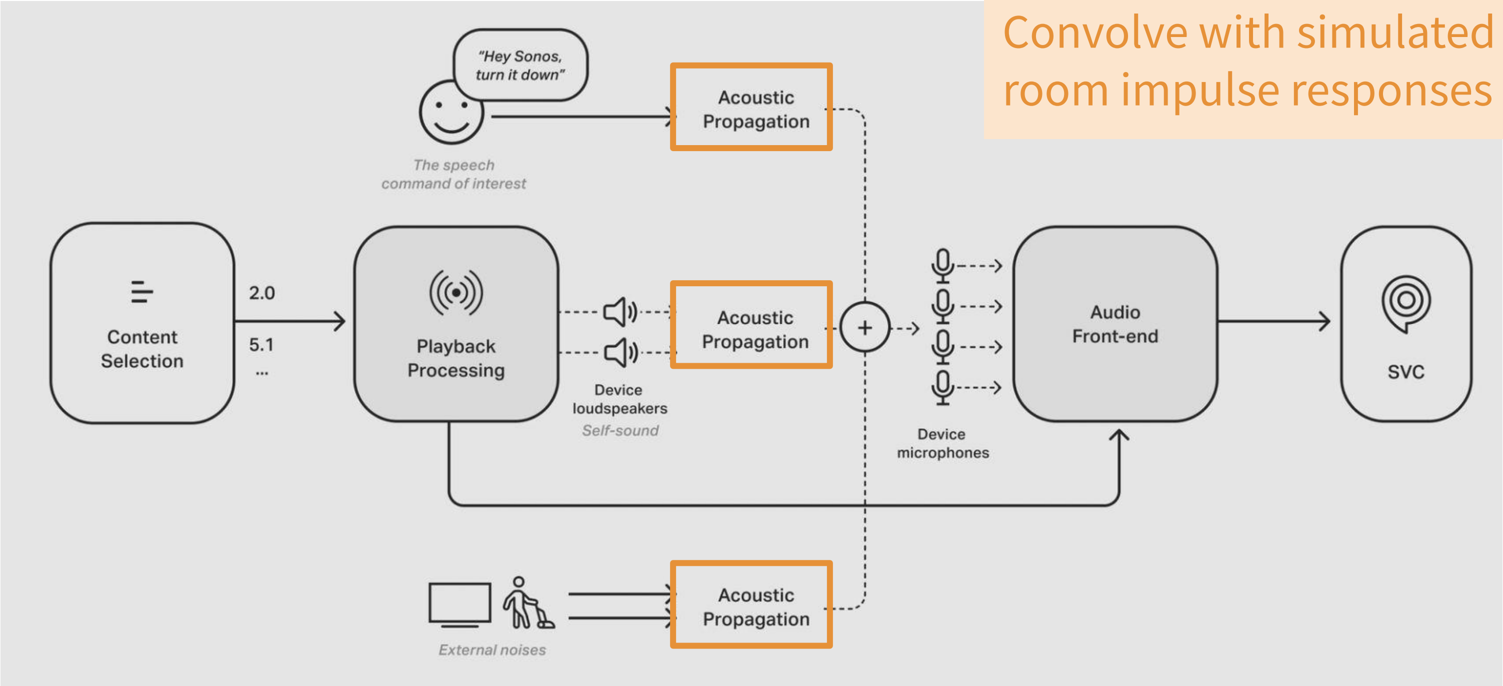
Spectral masking, beamforming

Adaptive filtering

Propagation to microphone(s) can be modeled with room impulse response(s)



Robust performance with data augmentation



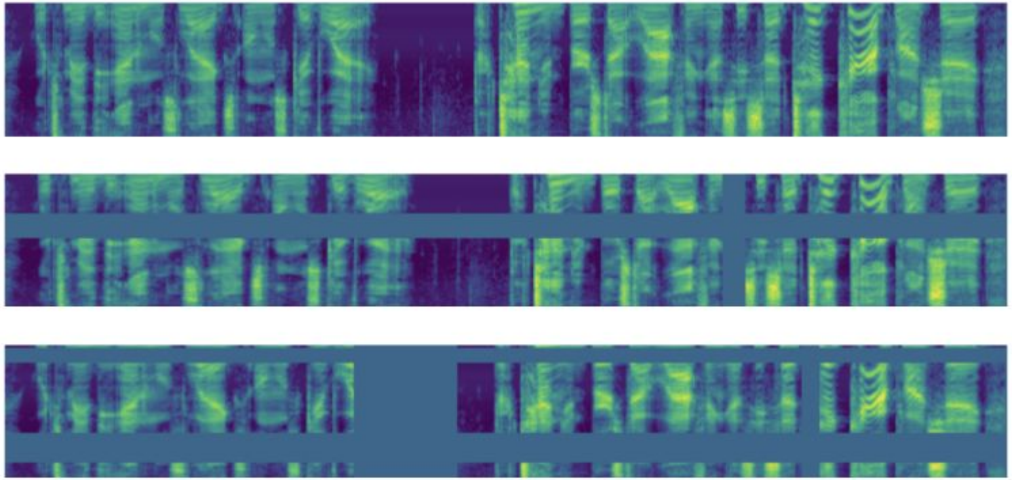
Convolve with simulated room impulse responses (RIRs).

[Data Augmentation for SVC]()

Simulating room impulse responses: https://github.com/LCAV/pyroomacoustics

More data augmentation

- Changing speed
- Changing pitch
- SpecAugment (time and frequency mapping)



Example code: https://speechbrain.readthedocs.io/en/latest/tutorials/preprocessing/speech-augmentation.html

Automatic Speech Recognition (Speech To Text)

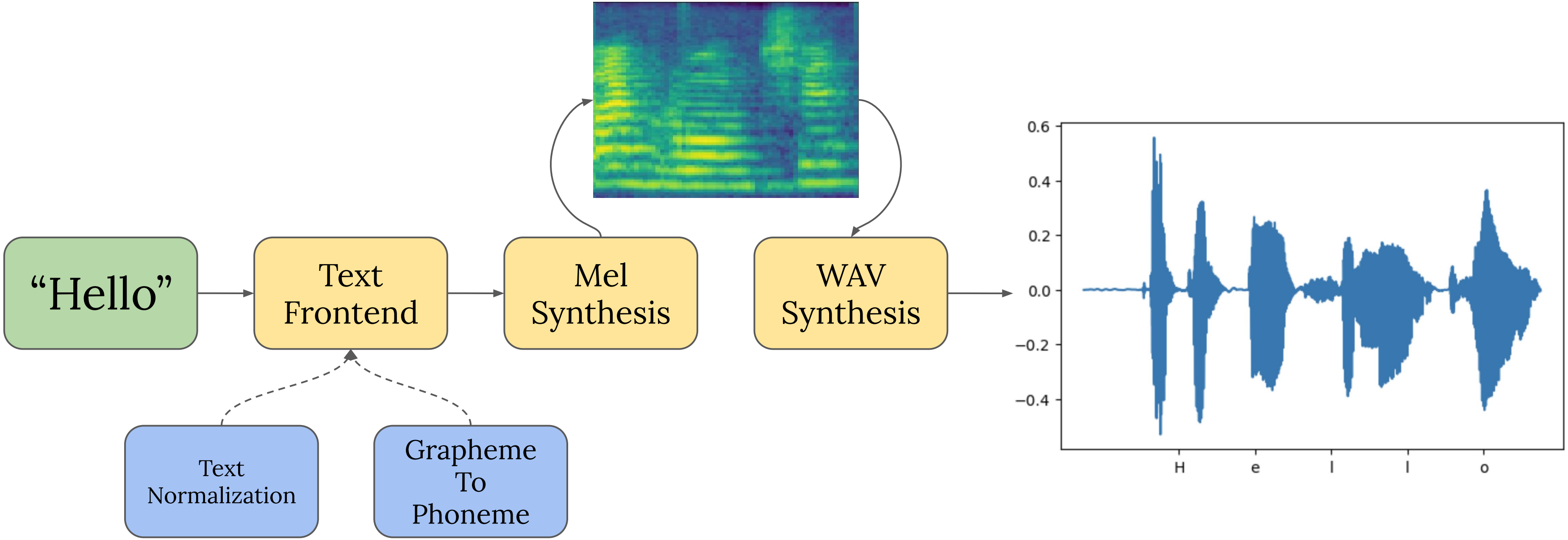
You can check HuggingFace Open ASR LeaderBoard and choose the model that fits your requirements

🏅 Leaderboard 📈 Metrics ✉️✨ Request a model here!

model	Average WER ⬇️	RTFx	AMI	Earnings22	Gigaspeech	LS Clean	LS Other
nvidia/canary-1b	6.5	235.34	13.9	12.19	10.12	1.48	2.93
nvidia/parakeet-tdt-1.1b	7.01	2390.61	15.87	14.49	9.52	1.4	2.6
nvidia/parakeet-rnnt-1.1b	7.12	2053.15	17.01	13.94	9.89	1.45	2.5
nvidia/parakeet-ctc-1.1b	7.4	2728.52	15.67	13.75	10.28	1.83	3.51
openai/whisper-large-v3	7.44	145.51	15.95	11.29	10.02	2.01	3.91
nvidia/parakeet-tdt_ctc-110m	7.49	5345.14	15.89	12.37	10.52	2.4	5.22
nvidia/parakeet-rnnt-0.6b	7.5	2815.72	17.4	14.66	10.01	1.62	3.02
distil-whisper/distil-large-v3	7.52	214.42	15.16	11.79	10.08	2.54	5.19
nvidia/parakeet-ctc-0.6b	7.69	4281.53	16.46	14.26	10.39	1.88	3.8

Text To Speech (TTS)

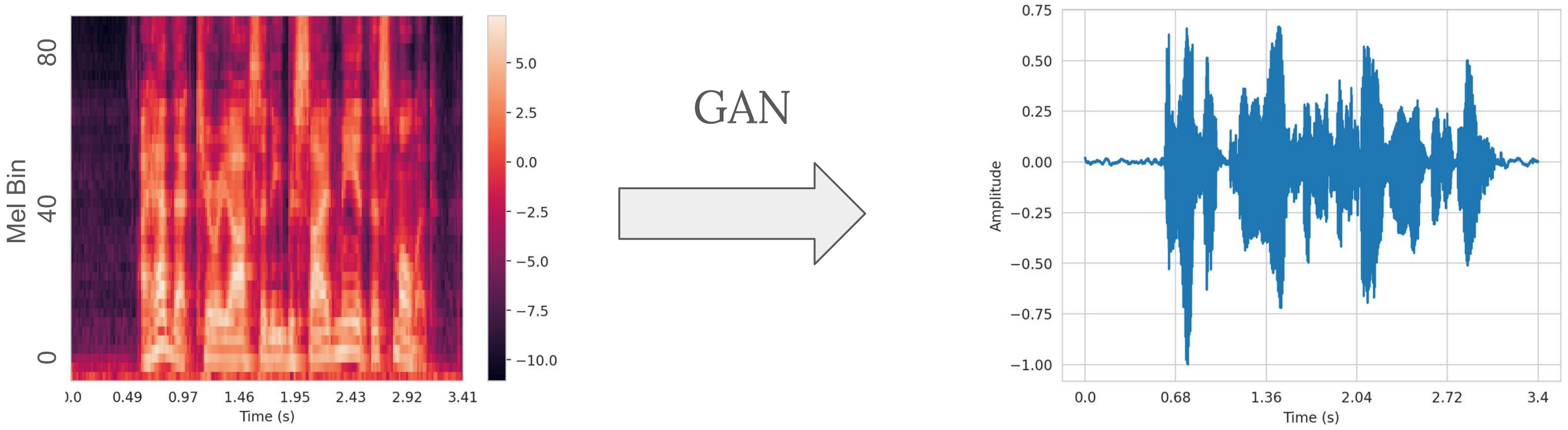
For TTS, most systems are divided into 2 parts: (i) Text-To-Intermediate Representation and (ii) Vocoder (Representation-To-Waveform)



Text To Speech - Wav Synthesis

Due to compression, Mel Spectrogram is not invertible (**lossy** transform)

We can use Generative Adversarial Networks to generate raw audio from Mel Spectrogram.

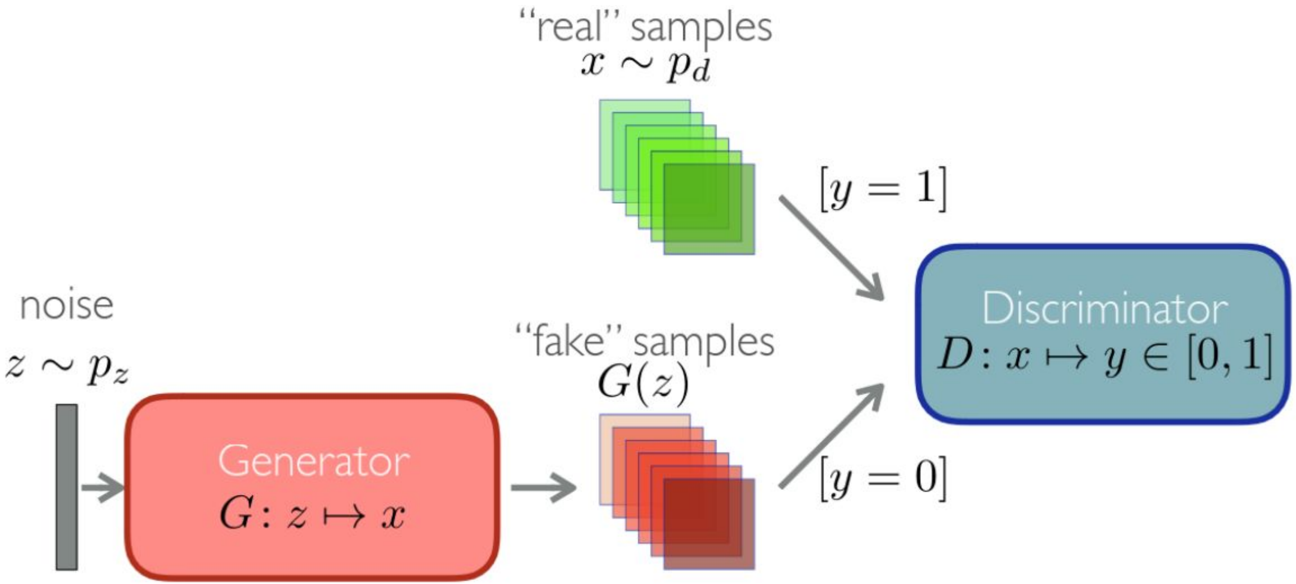


Remark: GANs

GAN consists of 2 networks: **Generator** and **Discriminator**

Discriminator solves binary classification task.

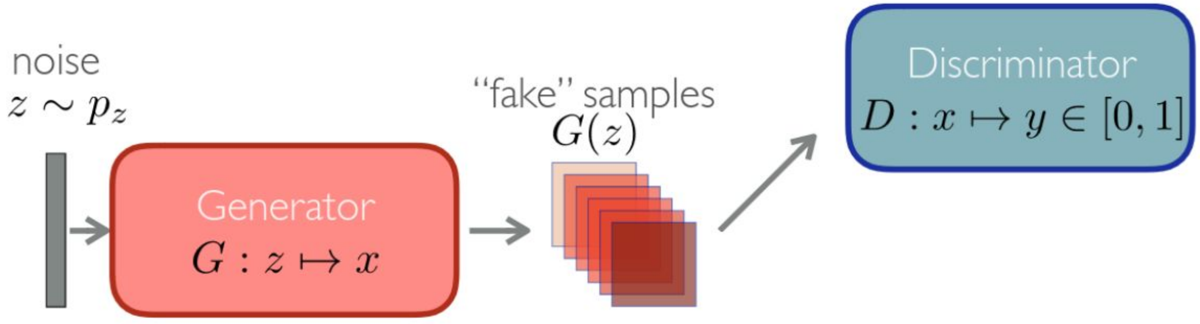
Given real and fake examples it predicts are they real or fake



Remark: GANs

GAN consists of 2 networks: **Generator** and **Discriminator**

Generator tries to generate realistic examples and fool the discriminator



x~p_g

Remark: GANs

There are alternative Loss Functions:
See [LSGAN] and [WGAN]

Two Networks play 2 players game:

$$\boxed{\min_G \max_D \mathop{\mathbb{E}}_{x\sim p_d}[\log D(x)] + \mathop{\mathbb{E}}_{z\sim p_z}[\log(1 - D(G(z)))]}$$

$$\mathscr{L}_D(G,D) = \max_D \mathop{\mathbb{E}}_{x\sim p_d}[\log D(x)] + \mathop{\mathbb{E}}_{z\sim p_z}[\log(1 - D(G(z)))]$$

$$\begin{aligned}\mathscr{L}_G(G,D) &= \min_G \mathop{\mathbb{E}}_{z\sim p_z}[\log(1 - D(G(z)))] \\ &:= \text{(in practice)} \ \max_G \mathop{\mathbb{E}}_{z\sim p_z}[\log(D(G(z)))]\end{aligned}$$

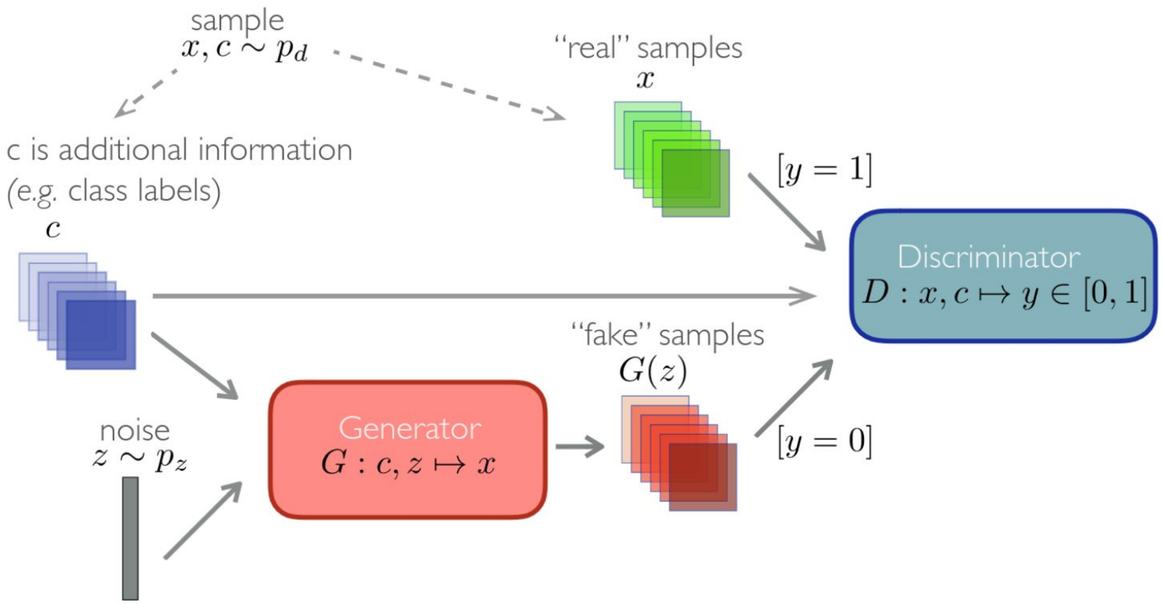
Remark: Training GANs

Train via alternating algorithm: Train Discriminator, then Generator

```
g_outputs = model.generate(**batch)
batch.update(g_outputs)

D_optimizer.zero_grad()

d_outputs = model.discriminate(generated_x=batch["generated_x"].detach(),
                                real_x=batch["real_x"])
batch.update(d_outputs)

D_loss = discriminator_criterion(**batch)
D_loss.backward()
D_optimizer.step()

G_optimizer.zero_grad()
d_outputs = model.discriminate(**batch)
batch.update(d_outputs)
G_loss = generator_criterion(**batch)
G_loss.backward()
```

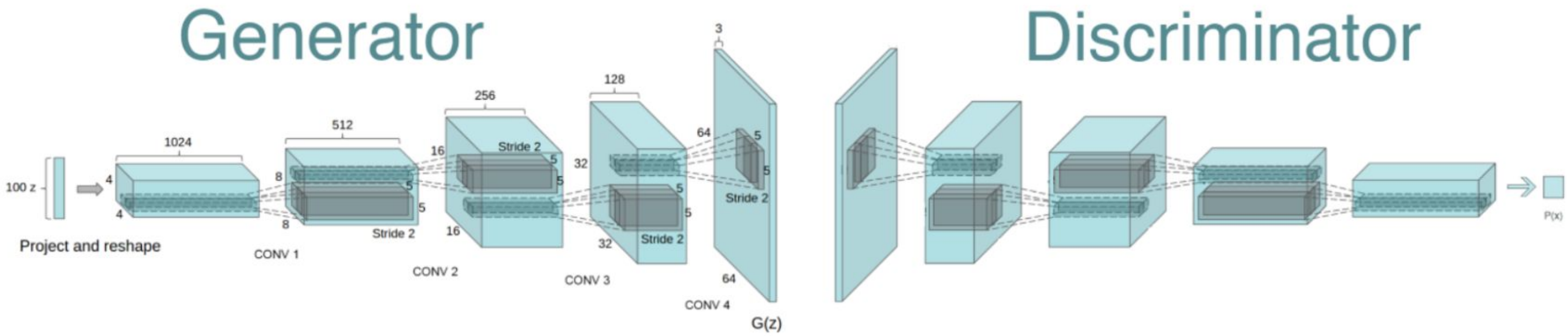
Remark: Conditional GANs (CGAN)

Conditional GANs have some other input to condition on apart from noise z



Remark: Deep Convolutional GAN (DCGAN)

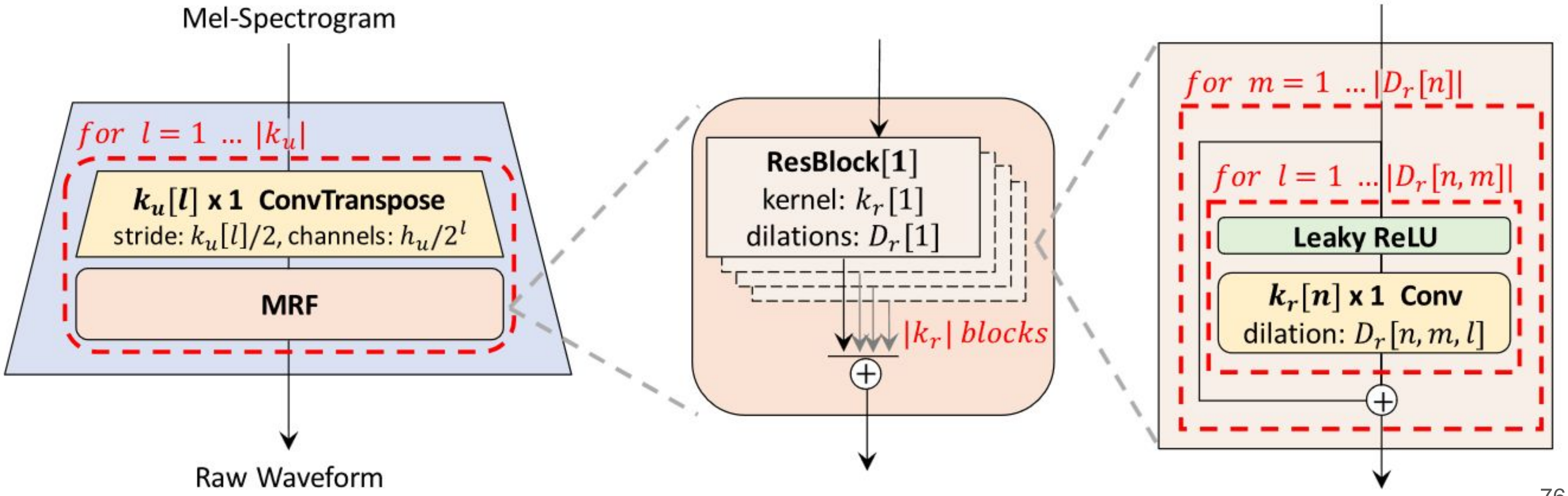
Classic architecture [DCGAN]() (Deep-Convolutional GAN)



HiFi-GAN (2020)

Generator converts Mel Spectrogram to a Waveform.

Many ResBlock with different Kernels/Dilations to model various receptive fields

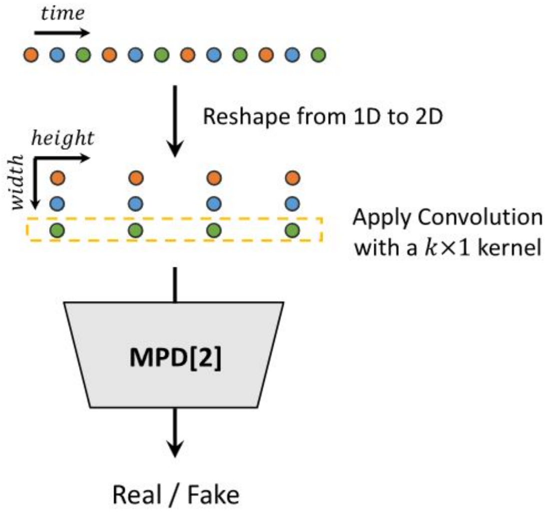


[HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis (NIPS 2020)]

HiFi-GAN (2020)

Multi-Period Discriminator analysis audio on different periods p [2, 3, 5, 7, 11]

This allows to look at diverse periodic patterns in the audio (recall that audio is a sum of sinusoids)

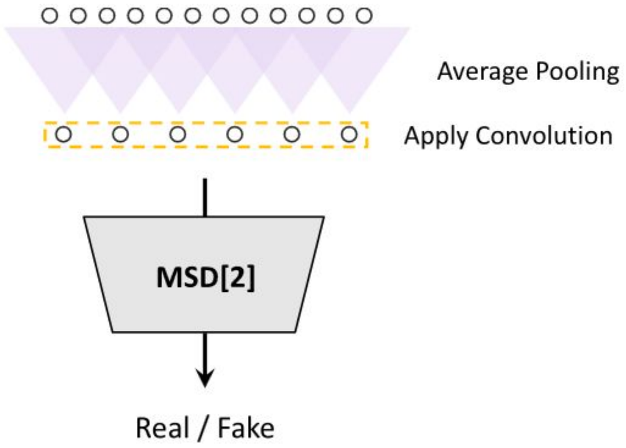


HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis (NIPS 2020)

HiFi-GAN (2020)

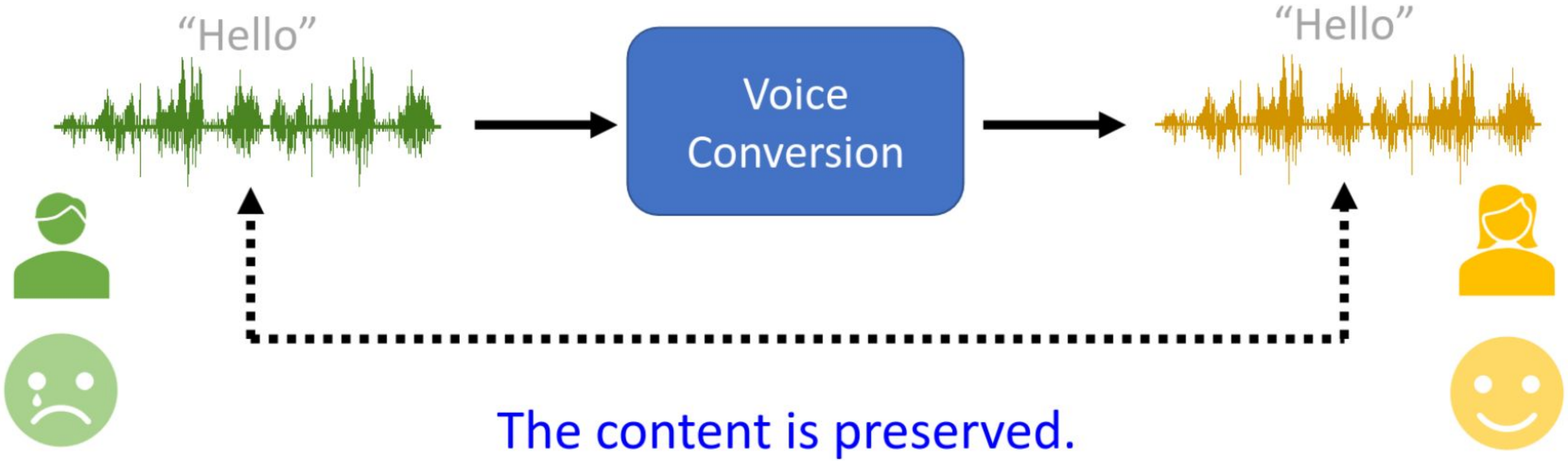
Multi-Scale Discriminator analysis audio on different levels with different receptive fields

This allows to capture consecutive patterns and long-term dependencies



[HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis (NIPS 2020)]

Voice Conversion (VC)



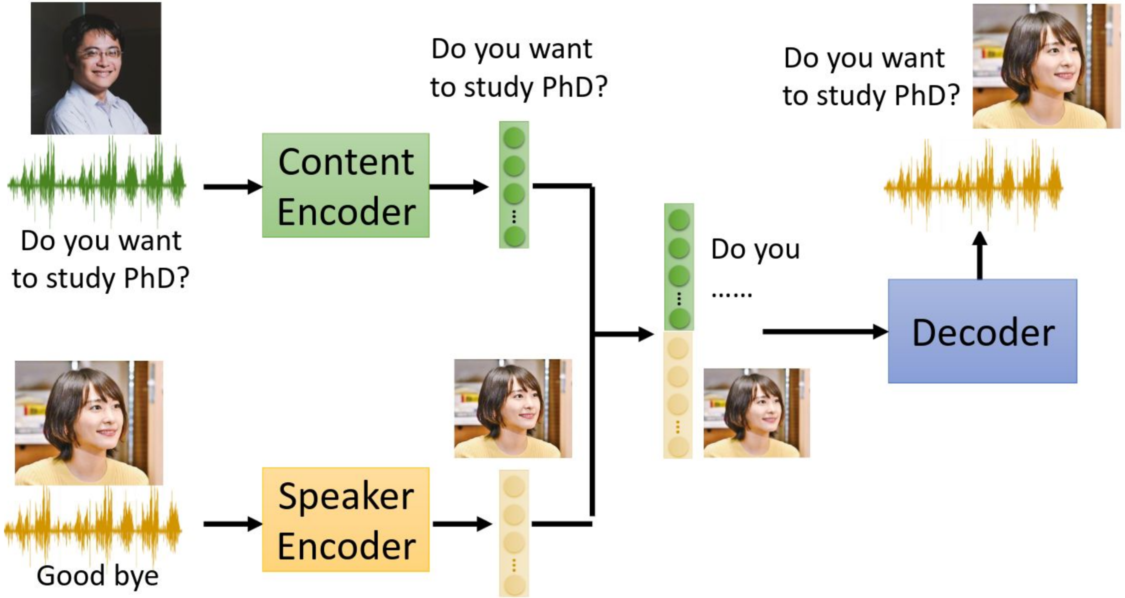
The content is preserved.

Many different aspects can be converted.

Voice Cloning – similar stuff, but the speaker is preserved, content is changed

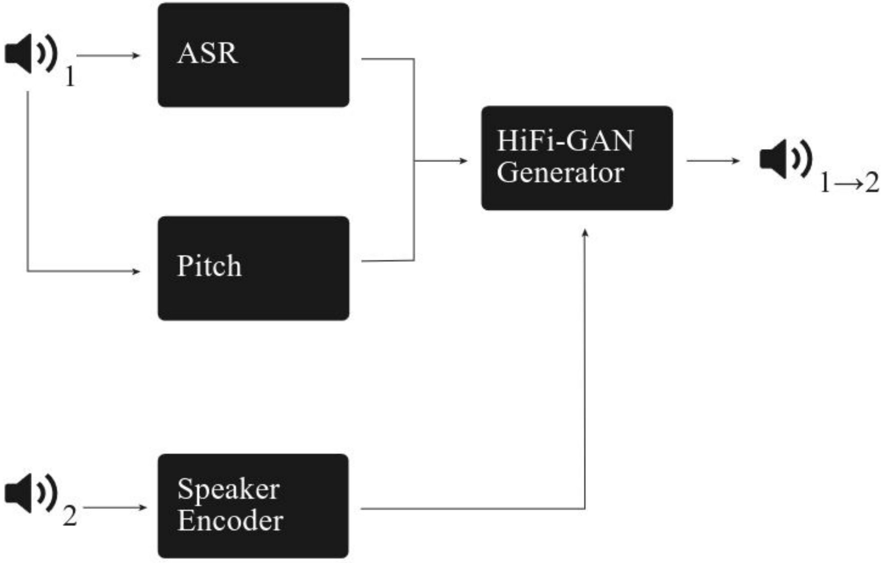
Voice Conversion (VC)

A common design approach is to have 2 networks. One to extract content information (what speaker says) and another one to extract speaker information (tembre, pitch, etc.). The outputs are combined to get a new audio with the content of the first speaker and identity of the second one



HiFi-VC (2023)

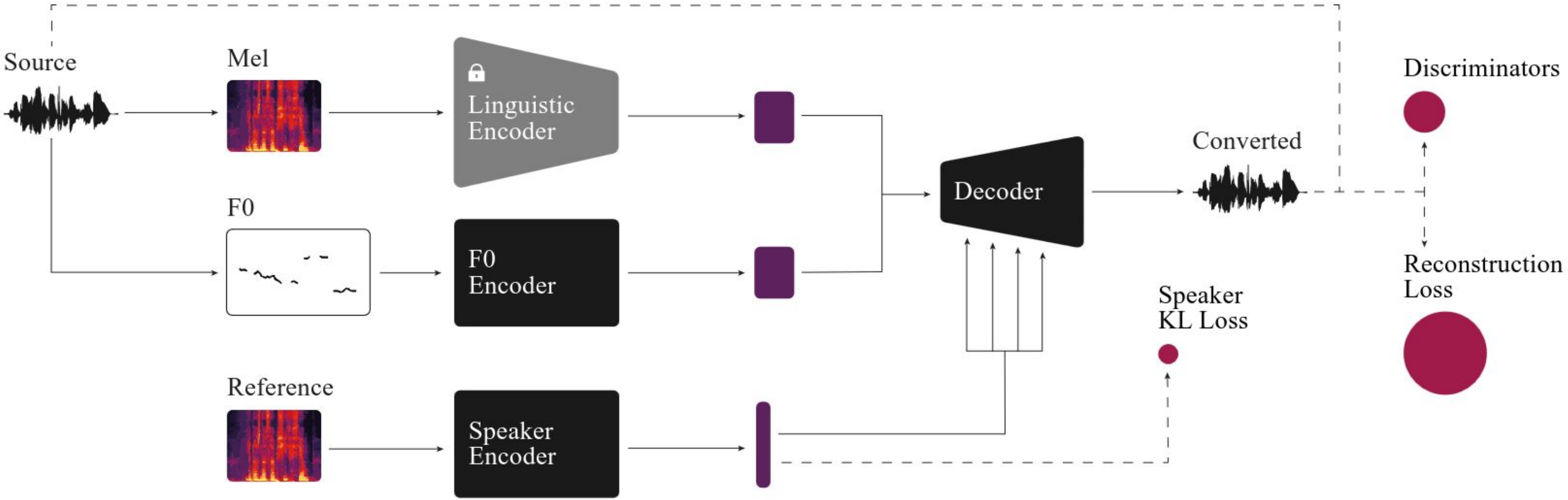
Combines HiFi-GAN from TTS, Encoder from ASR, VAE from Voice Biometry



HiFi-VC: High Quality ASR-Based Voice Conversion (Interspeech 2023)

HiFi-VC (2023)

Combines HiFi-GAN from TTS, Conformer from ASR, VAE from Voice Biometry



Want to know more about DLA?

See our HSE DLA Course and LauzHack Deep Learning Bootcamp

https://github.com/markovka17/dla

https://github.com/LauzHack/deep-learning-bootcamp

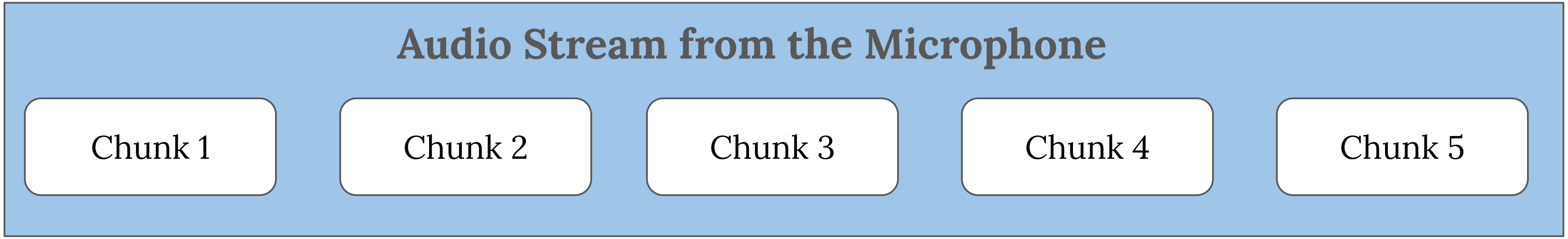
Let's Start Building An Assistant

The Voice Assistant Code

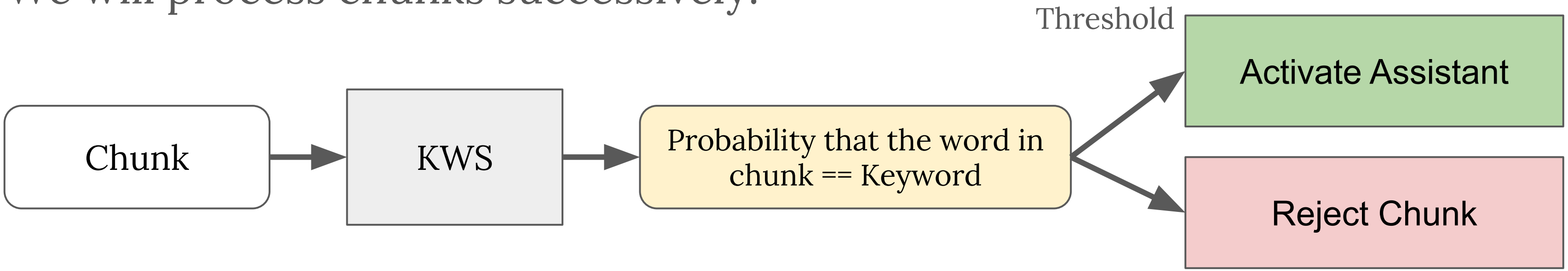
The code for the assistant is available via the link below. We will go through it right now.

The Voice Assistant Pipeline: KWS and Streaming

We start with a KWS system that will activate the assistant (like “Hey, Siri” or “Ok, Google”).



We will process chunks successively:



Reading from the microphone via PyTorch

To ensure fast processing, it is better to separate the main process where we apply our models from the process that reads the stream. For this, we will use *multiprocessing lib* and two processes.

```python
def init_stream(all_models, history, config):

    # initialize multiprocessing stuff
    ctx = mp.get_context("spawn")
    manager = ctx.Manager()
    chunk_queue = manager.Queue(maxsize=0)
    stream_init_queue = manager.Queue(maxsize=0)

    # initialize recording process
    streaming_process = ctx.Process(
        target=audio_stream,
        args=(
            stream_init_queue,
            chunk_queue,
            config.stream_reader.source,
            config.stream_reader.format,
            config.stream_reader.chunk_size,
            config.stream_reader.sample_rate,
            config.stream_reader.vad_limit,
        ),
    )

```

Reading from the microphone via PyTorch

The **Queue** object controls the communication between the processes. We will use it to send audio chunks from the microphone to the main process. We will also send control commands from the main process to the microphone one ("start recording", "stop recording")

```
def audio_stream(

    kws = get_kws()
    while init_queue.get() == SIG_START:  # main process sent the signal to start
        streamer = StreamReader(src=source, format=format)
        streamer.add_basic_audio_stream(
            frames_per_chunk=chunk_size, sample_rate=sample_rate, stream_index=0
        )
        stream_iterator = streamer.stream(timeout=-1, backoff=1.0)
```

Reading from the microphone via PyTorch

Read from the microphone stream

```
def audio_stream(

        print("Start audio streaming")
        while True:
            (chunk_,) = next(stream_iterator)
```

Sending chunk to the main process for further processing.

```
def audio_stream(

            # send chunk to the main process for processing
            # and output generation
            queue.put(chunk_data)
```

Sending command that we achieved recording stop criteria

```
def audio_stream(

        queue.put(SIG_STOP)  # stop criteria for the main process loop
        print("End of the user input")
        streamer.remove_stream(0)  # remove stream with id=0
```

Reading from the microphone via PyTorch

In the main process, we init the recording process, send the command to start the recording and get chunks from the recording process:

```
def init_stream(all_models, history, config):
    # start the recording process
    streaming_process.start()
    stream_init_queue.put(SIG_START)  # sent signal to start
    while True:
        try:
            while True:
                chunk = chunk_queue.get()
                if isinstance(chunk, int):
                    # stop criteria achieved
                    # chunk == SIG_STOP
                    break
                # concatenate successive query chunks
                user_chunks.append(chunk)
```

Reading from the microphone via PyTorch

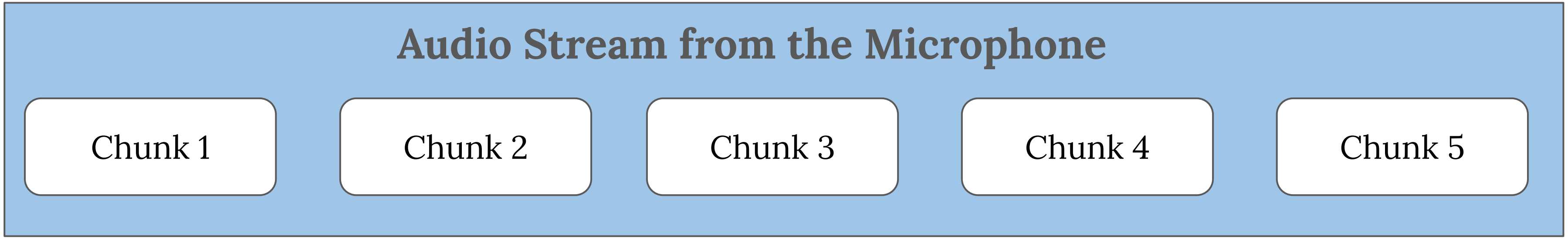
After processing the chunks (and using TTS to generate response), we send the signal to start recording again. In case we want to exit, we send exit code

```
            # init next user input
            user_chunks = []
            stream_init_queue.put(SIG_START)

        except KeyboardInterrupt:
            # stop the program
            break
        except Exception as exc:
            raise exc

    # send signal to exit
    stream_init_queue.put(SIG_STOP)
    streaming_process.join()
    stream_writer.close()
```

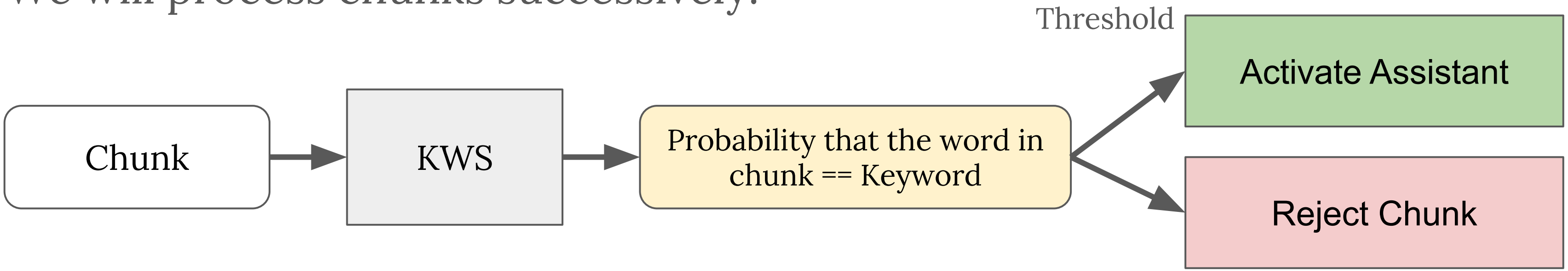
Join command ensures that we will wait for the end of the second process

The Voice Assistant Pipeline: KWS and Streaming

We start with a KWS system that will activate the assistant (like “Hey, Siri” or “Ok, Google).



We will process chunks successively:



The Voice Assistant Pipeline: KWS and Streaming

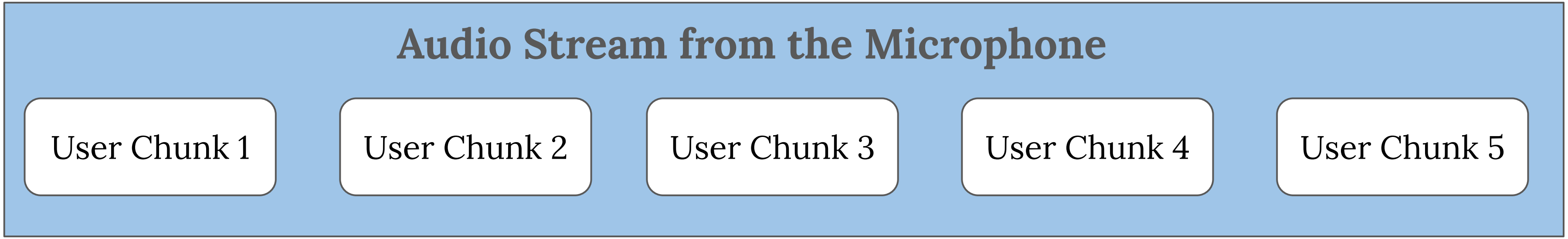
Ideally, the streaming process should only record audio, but for the simplicity we add some checks like KWS in it too.

```
print("Start audio streaming")
while True:
    (chunk_,) = next(stream_iterator)
    chunk_data = chunk_.data
    chunk_data = chunk_data.sum(dim=-1)  # to mono
    chunk_data = chunk_data.view(1, -1)

    if not query_started:
        # check that the query is started
        with torch.inference_mode():
            keyword_proba = kws(chunk_data)
        if keyword_proba > KWS_THRESHOLD:
            # the assistant is activated
            print(f"Keyword Detected. Probability: {keyword_proba}")
            query_started = True
        else:
            continue

    # send chunk to the main process for processing
    # and output generation
    queue.put(chunk_data)
```

The Voice Assistant Pipeline: VAD

After the Assistant is **activated**, we need to listen for the user's query. How to understand whether is it ended or not?

We will listen to all the chunks after assistant is activated



If the user is silent for several successive chunks, we will believe he ended the query. We can check silence via Voice Activity Detector

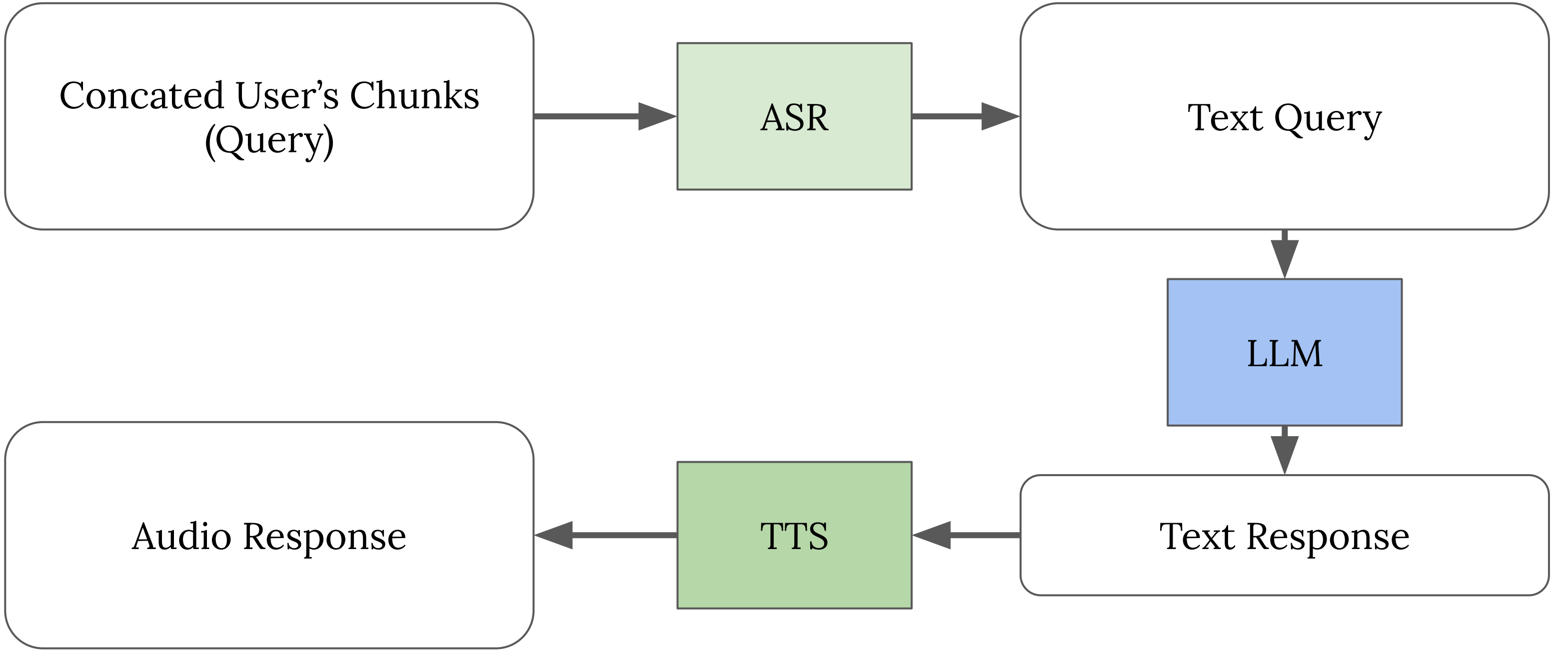
The Voice Assistant Pipeline: VAD

```python
def vad_checking(chunk, sample_rate):

    vad_chunk = vad(chunk, sample_rate=sample_rate)
    # some rule to identify if the chunk contains speech
    if vad_chunk.shape[1] <= chunk.shape[1] * SILENCE_PROPORTION_IN_CHUNK:
        return IS_SILENCE
    else:
        return IS_SPEECH

```

```python
def audio_stream(

            # check stop criteria: vad_limit successive silent chunks
            vad_check = vad_checking(chunk_data, sample_rate=sample_rate)
            if vad_check:
                vad_counts += 1
            else:
                vad_counts = 0
                user_talked = True
            if user_talked and vad_counts >= vad_limit:
                break
```

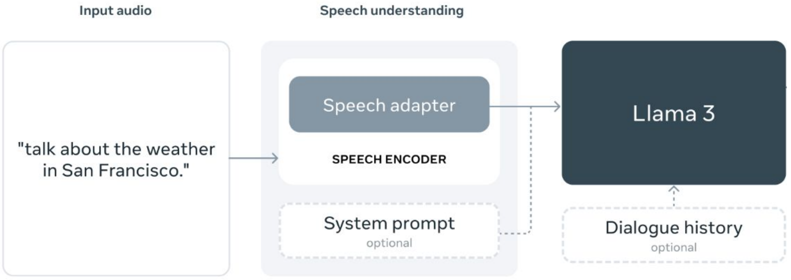
The Voice Assistant Pipeline: Getting Audio Response

With the help of VAD, we got the whole user's query. Now we need to generate a response:



Remark: Can we avoid ASR and feed the audio directly?

Novel LLMs are capable of working with Audio directly. However, the pipeline is almost the same, the model just skips explicit text representation. Example from LLAMA-3



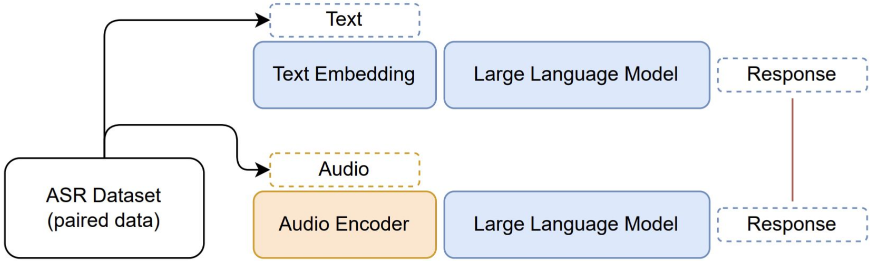


Figure 2: The aim is to create an end-to-end system that would generate the same response when being fed spoken (audio) input instead of its text version. The overall system should be invariant to the modality of the inputs containing the same semantic information.

AudioChatLlama: Towards General-Purpose Speech Abilities for LLMs (2024), The Llama 3 Herd of Models (2024)

Where to get models?

The first approach is to train a model yourself, like we did for KWS. The second option is to use pre-trained models

General Use: HuggingFace and GitHub contain many pre-trained models. For audio and many other domains.

Where to get models?

Audio-Specific: There are several libraries that provide pre-trained models specifically for audio domain (some of them have other domains too, but the focus is mostly on audio)

FAIRSEQ

NVIDIA NEMO

SpeechBrain

ESPnet

ASR and TTS: Defining an ASR model

We will use HuggingFace to get ASR and TTS models. Let’s start with ASR

```python
def init_asr_model(config):
    """
    torch_dtype = torch.float32 if config.device == "cpu" else torch.float16

    model = AutoModelForSpeechSeq2Seq.from_pretrained(
        config.asr.model_id,
        torch_dtype=torch_dtype,
        low_cpu_mem_usage=True,
        use_safetensors=True,
    )
    model.to(config.device)

    processor = AutoProcessor.from_pretrained(config.asr.model_id)

    asr_pipeline = pipeline(
        "automatic-speech-recognition",
        model=model,
        tokenizer=processor.tokenizer,
        feature_extractor=processor.feature_extractor,
        torch_dtype=torch_dtype,
        device=config.device,
    )
    return asr_pipeline
```

Model Id is the model name from the HuggingFace Hub. For example, *openai/whisper-tiny.en*

ASR and TTS: Running the ASR model

HuggingFace allows us to use pre-defined pipeline to simplify inference

```python
def run_asr_model(asr_pipeline, audio):
    """
    Get transcription for a speech input.

    Args:
        asr_pipeline: HF pipeline to get text from audio input.
        audio (torch.Tensor): input audio.
    Returns:
        text_output (str): text transcription of user's query.
    """
    # pipeline expects numpy array of shape (T)
    # so we convert and take 0-th channel
    text_output = asr_pipeline(audio.numpy()[0])["text"]

    return text_output
```

ASR and TTS: Defining a TTS model

For the ASR, we used base classes. We can use the direct ones, as we show for TTS

```python
def init_tts_model(config):
    tokenizer = FastSpeech2ConformerTokenizer.from_pretrained(
        "espnet/fastspeech2_conformer",
    )
    spec_generator = FastSpeech2ConformerModel.from_pretrained(
        "espnet/fastspeech2_conformer",
    )
    vocoder = FastSpeech2ConformerHifiGan.from_pretrained(
        "espnet/fastspeech2_conformer_hifigan",
    )

    spec_generator.to(config.device)
    vocoder.to(config.device)

    return tokenizer, spec_generator, vocoder
```

Remember that TTS consists of two parts:

1. Creating MelSpec
2. Converting it to audio

Different models can be used for 1 and 2.

ASR and TTS: Running the TTS model

In contrast to ASR, here we define the pipeline ourselves:

```python
# Generate audio
# text -> encoded_text -> mel spec -> waveform
inputs = tokenizer(query, return_tensors="pt")
input_ids = inputs["input_ids"]
output_dict = spec_generator(input_ids.to(config.device), return_dict=True)
spectrogram = output_dict["spectrogram"]

audio = vocoder(spectrogram).detach().cpu()
yield audio
```

LLM

For the LLM, running inference on your own machine is complicated (not impossible though). It is better to use APIs.

We could use well-known ChatGPT, but its API is not free

LLM

For the LLM, running inference on your own machine is complicated (not impossible though). It is better to use APIs.

We could use well-known ChatGPT, but its API is not free

The same interface but has free APIs. For example, for Meta's llama-3

LLM: init client

When we created an API key, we can start a LLM client:

```python
def init_llm():
    api_path = ROOT_PATH / ".env"
    with api_path.open("r") as f:
        api_key = f.readline().strip()
    client = Groq(api_key=api_key)

    history = [
        {
            "role": "system",
            "content": "You are an intelligent ai voice assistant named Sheila",
        }
    ]

    return client, history

```

Note that we need to handle history ourselves. We also start with a system message

LLM: run inference

For inference, we just add the new query and delete too old elements from the history

```python
def run_llm(text, history, client, config):
    Returns:
        predicted_text (str): LLM response.
        history (list[dict]): updated history.
    """
    max_tokens = config.llm.max_tokens
    max_history = config.llm.max_history

    if len(history) >= max_history + 1:
        history = [history[0]] + history[-(max_history - 1) :]

    history.append({"role": "user", "content": text})

    chat_completion = client.chat.completions.create(
        messages=history,
        model=config.llm.model_id,
        max_tokens=max_tokens,
    )

    predicted_text = chat_completion.choices[0].message.content

    history.append({"role": "assistant", "content": predicted_text})

    return predicted_text, history
```

Problem: Response may be loooooong

What is the LLM response is too long?

1. We cannot run TTS anymore (memory limitations)

2. Even if we can, generation of audio will take a lot of time. The user will wait in silence for a long period of time (it is bad)

Solution: split the response into several text partitions and generate audio for them individually

Solution

Split the text

```
def run_tts_model(text, tokenizer, spec_generator, vocoder, config):

    max_words_per_query = config.tts.max_words_per_query

    text_partitions = text.split()
    returned_partitions = 0
    number_of_partitions = len(text_partitions) // max_words_per_query
    for i in range(number_of_partitions + 1):
        # for a case when len(text_partitions) % max_words_per_query == 0
        if i * max_words_per_query == len(text_partitions):
            break
        query = text_partitions[i * max_words_per_query : (i + 1) * max_words_per_query]
        returned_partitions += len(query)

        query = " ".join(query)
```

Solution

Process part of the text individually and yield it:

```python
# Generate audio
# text -> encoded_text -> mel spec -> waveform
inputs = tokenizer(query, return_tensors="pt")
input_ids = inputs["input_ids"]
output_dict = spec_generator(input_ids.to(config.device), return_dict=True)
spectrogram = output_dict["spectrogram"]

audio = vocoder(spectrogram).detach().cpu()
yield audio
```

Writing to the loudspeaker

We could also create a separate process, but for the simplicity we are doing it in the main process.

```
def init_stream(all_models, history, config):

    # initialize writer for audio output playback
    stream_writer = StreamWriter(
        dst=config.stream_writer.dst, format=config.stream_writer.format
    )
    stream_writer.add_audio_stream(
        sample_rate=config.stream_writer.sample_rate, num_channels=1
    )
    stream_writer.open()

```

Writing to the loudspeaker

For each text partition, we get generated audio and play it chunk-by-chunk

```
def init_stream(all_models, history, config):
            # send it to ASR, LLM and TTS
            # get output audio generator
            user_full = torch.cat(user_chunks, dim=-1)
            user_audio_output_generator, history = full_handler(
                all_models, history, user_full, config
            )

            start_time = time.perf_counter()
            total_num_frames = 0
            for user_audio_output in user_audio_output_generator:
                user_audio_output = user_audio_output[0].unsqueeze(-1)
                num_frames = user_audio_output.shape[0]
                total_num_frames += num_frames

                # write the chunks to the StreamWriter for a playback
                for i in range(0, num_frames, config.stream_writer.chunk_size):
                    stream_writer.write_audio_chunk(
                        0, user_audio_output[i : i + config.stream_writer.chunk_size]
                    )

            audio_time = total_num_frames / config.stream_writer.sample_rate
            end_time = time.perf_counter()
```

Problem with writing

The process sends the chunks to the stream, but it does not know whether the loudspeakers have ended playing or not.

```
def init_stream(all_models, history, config):
            end_time = time.perf_counter()

            # to avoid reading from the microphone before the output is finished
            # we calculate the audio output time and the time we used to
            # process the audio. We stop the process for the difference time,
            # so the audio output finished before we move on.
            diff_time = audio_time - (end_time - start_time)
            if diff_time > 0:
                print(f"Audio time: {audio_time}. Have to sleep for {diff_time}.")
                # 1 extra second to be sure
                time.sleep(1 + diff_time)  # to avoid ASR reading an output

            # init next user input
            user_chunks = []
            stream_init_queue.put(SIG_START)
```

Solution: calculate the time of the audio response and make the main process sleep for these number of seconds.

Demo Time